HW4: Nonlinear and Ensemble Modeling (due 12/12 Monday 23:30)

Name: 莊明儒 Student ID: 111024513

Contents

1 Problem 1: Bike Sharing Dataset **2**

1.1 Train-test splitting . . . 3

1.2 EDA and preliminary findings . . . 3

1.3 Fit tree-based models and summarize their prediction performance. The tuning parameters for tree-based models should be determined via cross validation. . . . 12

1.4 Fit a linear regression and a nonlinear regression model for the target variable **count**. Again, your models should go through a proper model selection procedure. Summarize the selected models with their prediction performance. . . . 21

1.5 Evaluate the prediction performance via testing error on testing set . . . 28

1.6 Brief analysis summary . . . 29

2 Problem 2: Airline Customer Satisfaction **30**

2.1 Train-test splitting . . . 31

2.2 Brief EDA . . . 31

2.3 Find a good model for the classification of customer satisfaction, and summarize the performance. 38

2.4 Evaluate the prediction performance via testing error on testing set . . . 46

2.5 Identify the important aspects of the flight experience related to customer satisfaction (or dis-satisfaction). Based on your findings, give some specific suggestions (provided with the data evidence) to the airline company for achieving a higher customer satisfaction. . . . 47

2.6 Add some noise factors (both continuous variable and categorical variable) in your data set (call these added variables z1-z?), and check the ranking of variable importance of these z-variables relative to the original meaningful input variables. Make your comments regarding this experiment. . . . 48

3 Appendix **52**

3.1 A. Summary for the variables in Problem 2 . . . 52

1 Problem 1: Bike Sharing Dataset

This data set collects the bike sharing counts aggregated on daily basis (2011-2012) together with the calender information (year/season/weekday/holiday) and daily weather conditions. There are 4 weather related variables:

- weather: (GOOD, MISTY, RAIN/SNOW/STORM)
- temperature (in Celsius)
- humidity (relative to 100%)
- windspeed

The objective is to predict the bike sharing counts given a date, and figure out the impacts on the counts due to weather conditions.

More data descriptions can be found at http://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset

1.0.1 Your analysis should include the following:

- EDA and preliminary findings.
- Fit tree-based models and summarize their prediction performance. The tuning parameters for tree-based models should be determined via cross validation.
- Fit a linear regression and a nonlinear regression model for the target variable **count**. Again, your models should go through a proper model selection procedure. Summarize the selected models with their prediction performance.
- Identify the important input variables for predicting bike sharing counts. In particular, characterize the effects of weather conditions on the bike counts **quantitatively**.
- Give a brief analysis summary.

```
bike <- read.csv(file="bike.csv") #read data
head(bike)
```

```
##   days_since_2011 season year month    holiday weekday     workingday weather
## 1               1 WINTER 2011   JAN NO HOLIDAY     SAT NO WORKING DAY   MISTY
## 2               2 WINTER 2011   JAN NO HOLIDAY     SUN NO WORKING DAY   MISTY
## 3               3 WINTER 2011   JAN NO HOLIDAY     MON    WORKING DAY    GOOD
## 4               4 WINTER 2011   JAN NO HOLIDAY     TUE    WORKING DAY    GOOD
## 5               5 WINTER 2011   JAN NO HOLIDAY     WED    WORKING DAY    GOOD
## 6               6 WINTER 2011   JAN NO HOLIDAY     THU    WORKING DAY    GOOD
##       temp     hum windspeed count
## 1 8.175849 80.5833 10.749882   985
## 2 9.083466 69.6087 16.652113   801
## 3 1.229108 43.7273 16.636703  1349
## 4 1.400000 59.0435 10.739832  1562
## 5 2.666979 43.6957 12.522300  1600
## 6 1.604356 51.8261  6.000868  1606
```

```r
#View(bike)

cols = colnames(bike)
bike$season = factor(bike$season, levels = c("SPRING", "SUMMER", "FALL", "WINTER"))
bike$month = factor(bike$month, levels = c("JAN", "FEB", "MAR", "APR", "MAY",
                                           "JUN", "JUL", "AUG", "SEP", "OCT",
                                           "NOV", "DEC"))
bike$weekday = factor(bike$weekday, levels = c("MON", "TUE", "WED", "THU", "FRI",
                                               "SAT", "SUN"))
bike$holiday = factor(bike$holiday, levels = c("NO HOLIDAY", "HOLIDAY"))
bike$weather = factor(bike$weather, levels = c("RAIN/SNOW/STORM", "MISTY", "GOOD"))
bike$workingday = factor(bike$workingday)
bike$year = factor(bike$year)
```

1.1 Train-test splitting

To evaluate the prediction performance of each fitted model throughout the analysis, the data is splitted into training set and testing set. The training set and the testing set respectively accounts for 80% (585 customers) and 20% (146 observations) of the original data. The splitting is determined by simple random sampling without replacement (with random seed = 10).

```r
train = rep(FALSE, nrow(bike))
set.seed(10)
train[sample(1:nrow(bike), round(nrow(bike) * 0.8))] = TRUE
test = !train
```

Notice that throughout the study, every procedure involved with variable selection (including EDA) or model fitting will be conducted only on the training set, and hence no information of testing set would be misused.

```r
## Use information from the training set only!
bike.test = bike[test,]
bike = bike[train, ]
```

1.2 EDA and preliminary findings

1. Data description

This data set consists of 731 observations on 12 variables. According to the problem description, count is defined as the response variable and the remaining variables including season, year, month, holiday, weekday, working day, weather, temp (temperature), hum (humidity) and wind speed are defined as the predictors. The variable "days_since_2011" may not be regarded as a predictor in the following study and instead we only regard it as time index.

2. Summary for the variables

The response variable count is approximately continuous. Among the predictors, season, year, month, holiday, weekday, working day and weather are discrete, and temp, hum and wind speed are continuous. Table , 1, 2, 3 and 4 shows the counts of data for each level of those discrete variables, and Talbe 5 shows the five-number summary and means for those continuous variables.

```r
count(bike$season, bike$year, cols[2], cols[3])
```

Table 1: Summary for season and year

	season				year	
	SPRING	SUMMER	FALL	WINTER	2011	2012
counts	148	148	144	145	299	286

```
count(bike$month, c(), cols[4], c())
```

Table 2: Summary for month and

	month											
	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
counts	52	41	53	53	49	39	51	49	48	53	50	47

```
count(bike$holiday, bike$weather, cols[5], cols[8])
```

Table 3: Summary for holiday and weather

	holiday		weather		
	NO HOLIDAY	HOLIDAY	RAIN/SNOW/STORM	MISTY	GOOD
counts	568	17	18	206	361

```
count(bike$weekday, bike$workingday, cols[6], cols[7])
```

Table 4: Summary for weekday and workingday

	weekday							workingday	
	MON	TUE	WED	THU	FRI	SAT	SUN	NO WORKING DAY	WORKING DAY
counts	82	90	85	86	80	83	79	179	406

```
stat = round(sapply(bike[9:12], summary),2)
kable(as.data.frame(stat), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "", caption = "Summary for the continuous variables") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

According to the tables above, several important findings are summarized as follows.

1. We can see that the data points distribute quite uniformly amond the levels for all the discrete variables, except for holiday, working day and weather.
2. There are very few observations on holiday and very a lot on non-holiday. On the contrary, there are a lot observations on working day and few on non-working day, amount of which is almost the half of that of working days.

Table 5: Summary for the continuous variables

	temp	hum	windspeed	count
Min.	-5.22	0.00	1.50	22.00
1st Qu.	7.78	52.25	9.04	3071.00
Median	14.91	63.08	12.17	4458.00
Mean	15.18	63.13	12.79	4459.23
3rd Qu.	22.55	73.54	15.63	5892.00
Max.	32.50	97.25	34.00	8714.00

3. Most of the days (463 days) are with good weather. Some (247 days) are with misty weather, and very few (21 days) are with horrible weather, for example, rainy, snowy and storming.

4. The distributions for temp, hum, wind speed and count all seem to be quite symmetric though the maximum of wind speed and the minimum of count might look a bit unusual.

3. Univariate distribution and pair-wise relation among the variables

a. Univariate distribution

```
par(mfrow = c(1, 4))
for(i in 9:12){
  hist(bike[,i], xlab = cols[i], main = "")
}
```

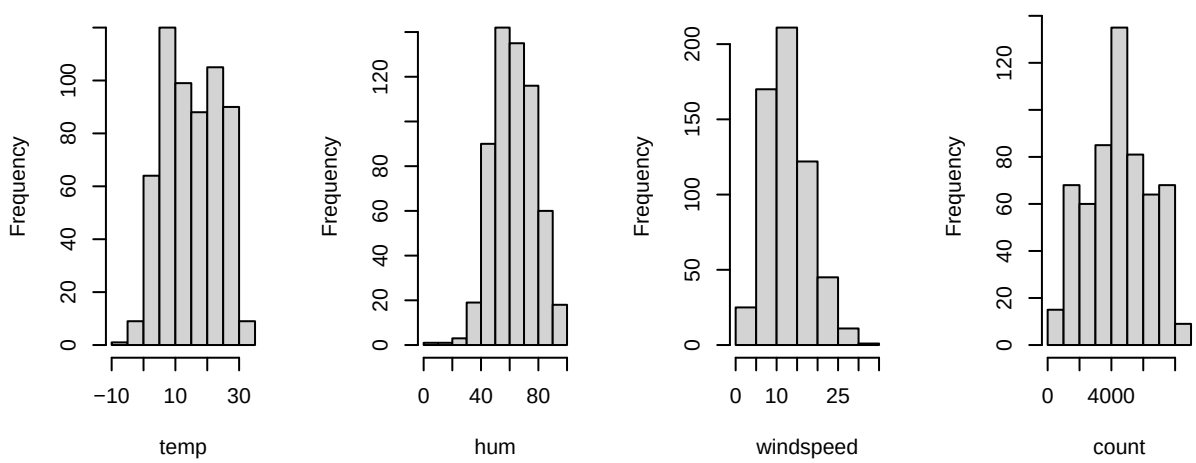

Figure 1: Histograms for the continuous variables

Figure 1 shows the univariate histograms for the continuous variables. We can see that they all look very symmetric though some may suspect the wind speed being somewhat skewed to the right. Overall, these histograms display consistent results with previous.

b. Pairwise relation

```
ggpairs(bike, columns = 2:12,
        upper = list(continuous = wrap("cor", size = 2)),
        lower = list(continuous = wrap("points", size=0.2))) %>%
        + theme(strip.placement = "outside", text = element_text(size = 8)) %>%
        + theme_bw()
```

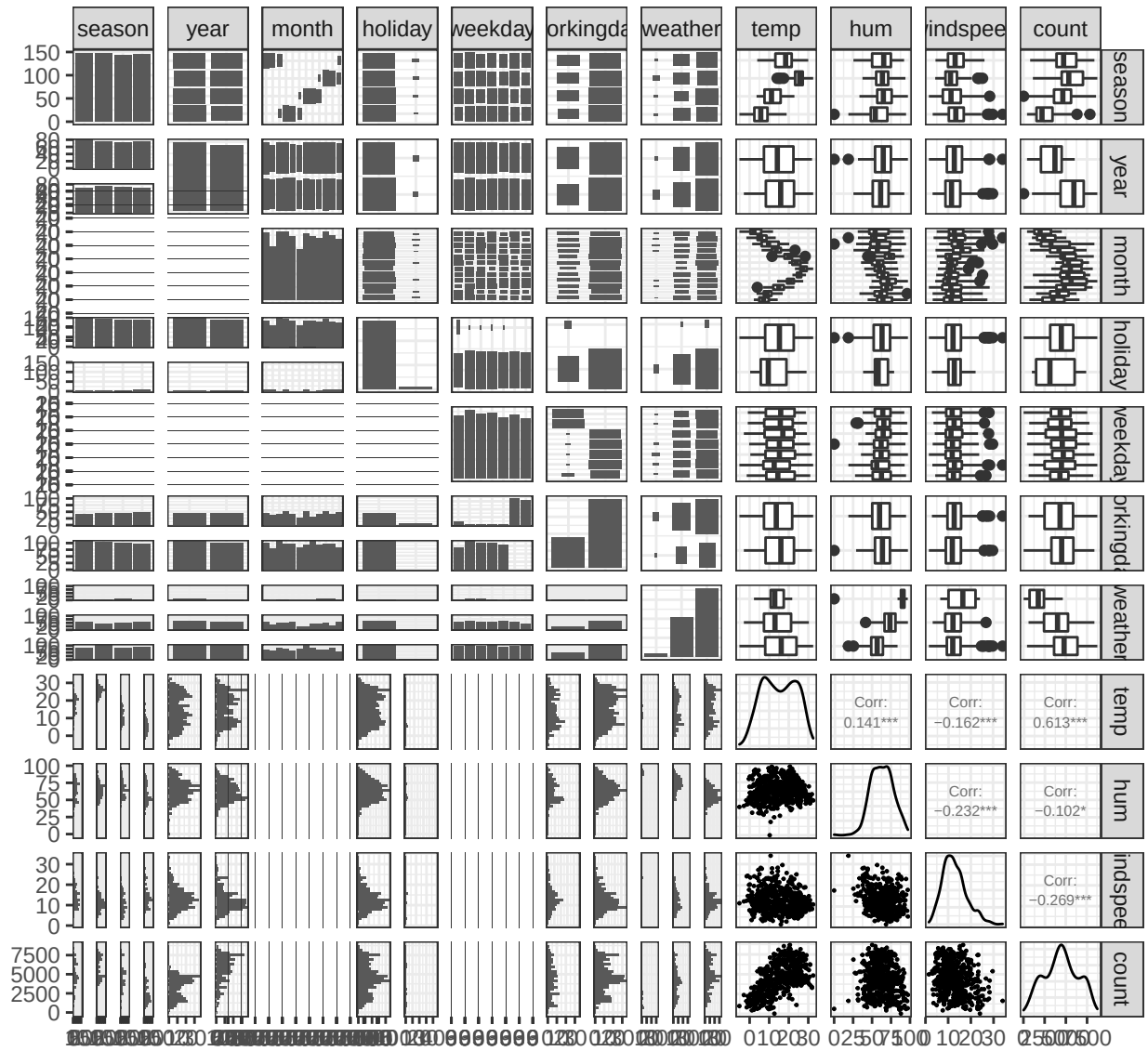

Figure 2: Pairwise relation among the variables

```
par(mfrow = c(1,3))
plot(temp~weather, bike, cex.axis = 0.8)
plot(hum~weather, bike, cex.axis = 0.8)
plot(windspeed~weather, bike, cex.axis = 0.8)
```

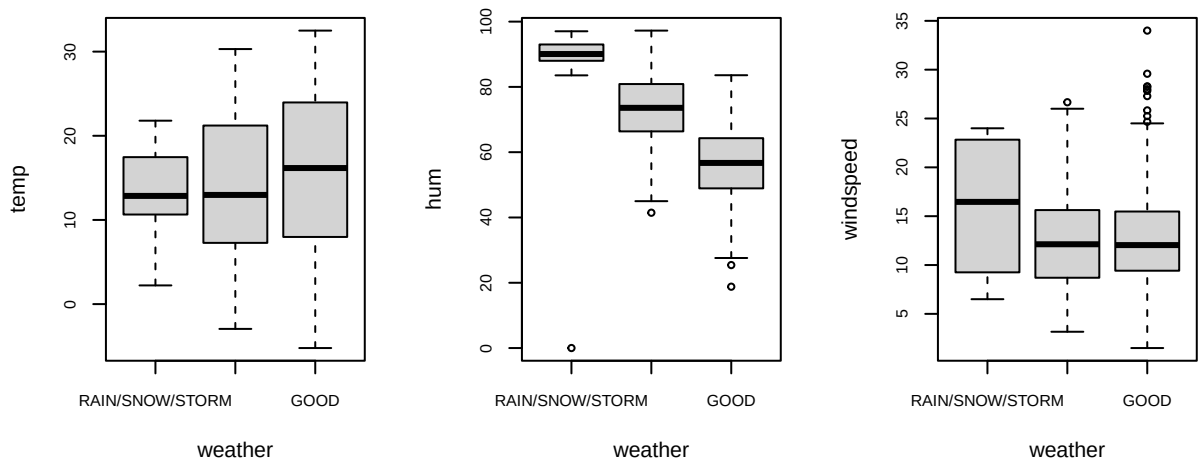

Figure 3: Relation between weather and related variables

Figure 2 shows the pairwise relation among the variables, and Figure 3 shows more detail about the relation between weather and temp, hum, and wind speed. According to them, several important findings are summarized as follows.

1. The continuous predictors all seem to be associated with count (actually, they are also statistically significant). To make it specific, temp and count are positively associative with correlation coefficient 0.627, suggesting medium level of association. On the contrary, hum and wind speed are negatively associated with count though there coefficient seems to be small, suggesting low level of association.

2. It seems that the better the weather, the more counts of bike sharing, which is quite reasonable.

3. No significant difference of counts of bike sharing seems to appear between working and non-working day. It might suggest that people use bike sharing in the same level no matter on working or on non-working day.

4. No significant difference of counts of bike sharing seems to appear among the weekdays. Such finding is consistent with the last one in the sense that working day is basically from Monday to Friday and non-working day is basically the weekend. In other words, working day and weekday are two variables sharing almost the same information.

5. People seems to use bike sharing less often on holidays than on non-holidays though such difference might not be significant due to the large variation on non-holidays (notice that we do not have many data on non-holiday).

6. The temperature seems to be highly associated with month and season. Such finding is quite trivial since we have hot weather in spring and summer and cold weather in fall and winter.

7. Overall, people seems to use bike sharing more often in spring and summer than in fall and winter. This finding may be due to the fact that temperature is highly associated with season, and temperature seems to have impact on bike sharing use.

8. People seems to use bike sharing more often in 2012 than in 2011. This finding might be due to the fact that as longer the bike sharing system has been under operation, people become more and more familiar with it and think it as reliable and thereby use the bike sharing system more often.

9. The temperature does not seem to differ a lot among the levels of weather though there seems to be difference on the humidity and the wind speed among different levels of weather. To make it specific, the humidity seems to be lower under better weather and the wind speed seems to be somewhat higher under bad weather such as rainy, snowy or storming.

10. There does not seem to be severe collinearity among the continuous predictor though the coefficients are statistically significant.

4. Time-dependent pattern

Due to the temporal nature of the data, it might be worth seeing if there is any time-dependent pattern in each variable. Figure 4 and 5 shows the interaction effects of each weather-related predictor and each time-related predictor by which we can see whether the impact of weather conditions on count depends on time.

```
m = matrix(1:15, nrow = 5, ncol = 3, byrow = TRUE)
layout(mat = m, height = c(0.25, 1, 1, 1, 1))
par(mar = c(1,1,1,1)+0.1)
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
par(mar = c(4,4,1,2)+0.1)
for(i in 9:(length(cols)-1)){
  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$year))){
    points(bike[bike$year==levels(bike$year)[j],i],
           bike[bike$year==levels(bike$year)[j],]$count, col = cbp[j], cex = 0.6)
  }

  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$season))){
    points(bike[bike$season==levels(bike$season)[j],i],
           bike[bike$season==levels(bike$season)[j],]$count, col = cbp[j], cex = 0.6)
  }

  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$month))){
    points(bike[bike$month==levels(bike$month)[j],i],
           bike[bike$month==levels(bike$month)[j],]$count, col = cbp[j], cex = 0.6)
  }
}
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend(x = "top", legend = levels(bike$year), cex = 0.7,
        col=cbp[1:length(levels(bike$year))], pch = 1, xpd = TRUE, bty = "n")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend(x = "top", legend = levels(bike$season), cex = 0.7,
        col=cbp[1:length(levels(bike$season))], pch = 1, xpd = TRUE, bty = "n")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend(x = "top", legend = levels(bike$month), cex = 0.6,
        col=cbp[1:length(levels(bike$month))], pch = 1, xpd = TRUE, bty = "n")
mtext("Interaction with year", side = 3, outer = TRUE, line = -2, cex = 1, at = 0.185)
mtext("Interaction with season", side = 3, outer = TRUE, line = -2, cex = 1, at = 0.515)
mtext("Interaction with month", side = 3, outer = TRUE, line = -2, cex = 1, at = 0.845)
```

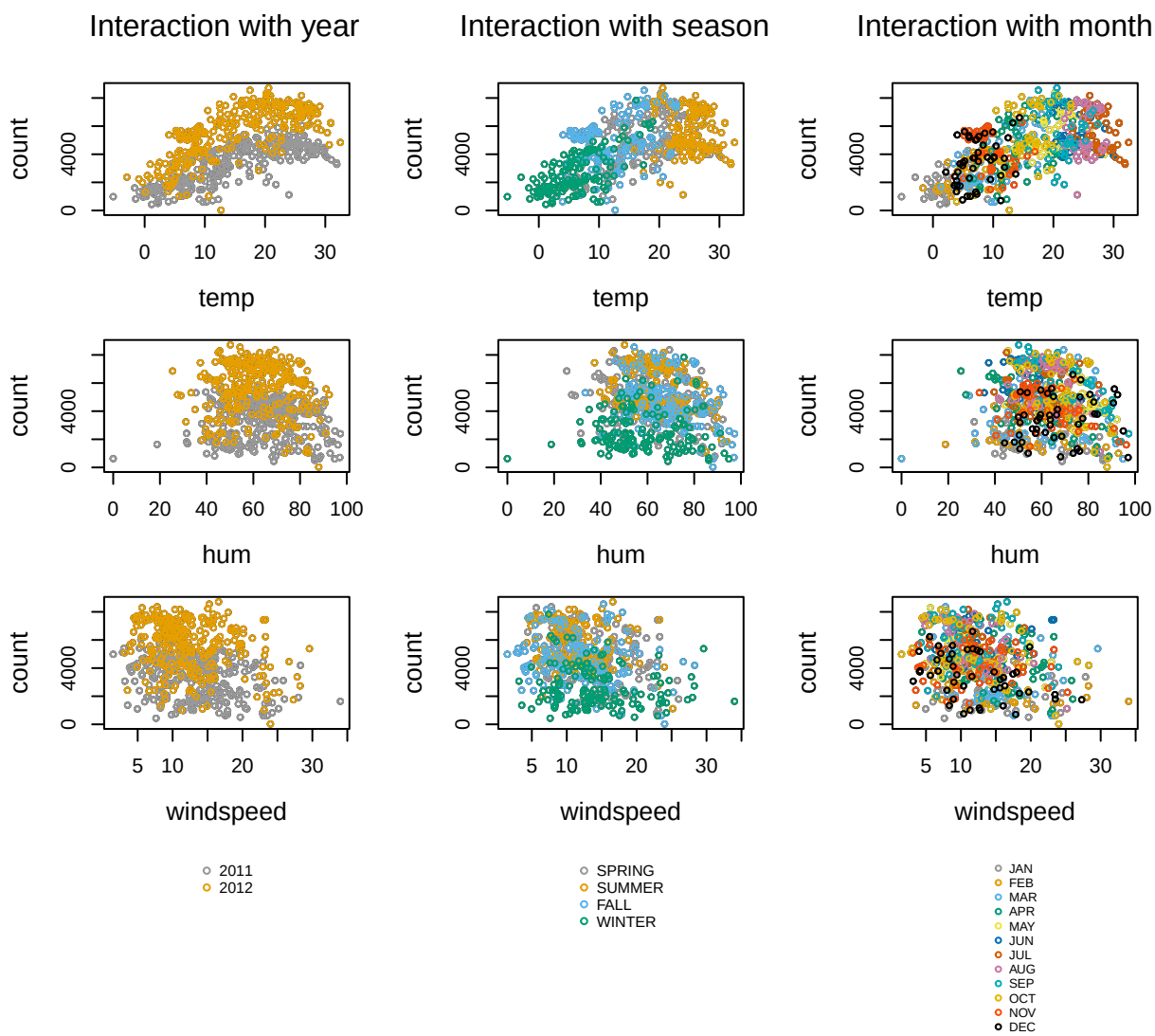

Figure 4: Time-dependent impact on count due to weather conditions

```
m = matrix(c(rep(1, 4), 2:5), nrow = 2, ncol = 4, byrow = TRUE)
layout(mat = m, heights = c(0.1, 1), widths = c(1, 1, 1, 0.25))
par(mar = c(0.5,0.5,0.5,0.5)+0.1)
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
par(mar = c(5,4,1,2)+0.1)
for(i in 9:(length(cols)-1)){
  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$weekday))){
    points(bike[bike$weekday==levels(bike$weekday)[j],i],
           bike[bike$weekday==levels(bike$weekday)[j],]$count, col = cbp[j], cex = 0.6)
  }
}
par(mar = c(0.5,0.5,0.5,0.5)+0.1)
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
```

```
legend(x = "top", legend = levels(bike$month), cex = 0.6,
        col=cbp[1:length(levels(bike$month))], pch = 1, xpd = TRUE, bty = "n")
mtext("Interaction with weekday", side = 3, outer = TRUE, line = -1, cex = 1, at = 0.48)
```

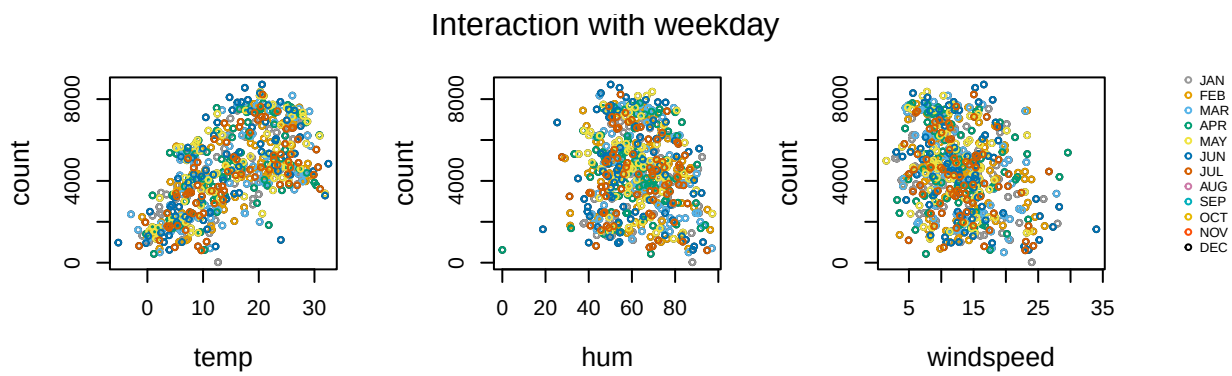

Figure 5: Time-dependent impact on count due to weather conditions (Cont.)

According to the scatter plots above, it seems that only the impact of temperature on count might depend on season and month. More specifically, in summer and the corresponding months, the increase in temperature seems to associate with decrease in the count of bike sharing, which is an opposite pattern than what would be observed in the other seasons or months.

Furthermore, by conditioning on the different years, it seems to become more clear that there might exist quadratic effect on count due to temperature. As one can see, the count seems to increase gradually as temperature climb up from quite low to medium (around $20^{\circ}C$). It seems to flatten a little bit over there ($20^{\circ}C$) and then decline as temperature climb continuously.

5. Day-type-dependent pattern

It might also be worth seeing whether there is any interaction between weather conditions and the day-type variables, including holiday and working day. Figure 6 shows the interaction effects of each weather-related predictor and each day-type-related predictor by which we can see whether the impact of weather conditions on count depends on day type.

```
m = matrix(1:10, nrow = 5, ncol = 2, byrow = TRUE)
layout(mat = m, height = c(0.25, 1, 1, 1, 1))
par(mar = c(1,1,1,1)+0.1)
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
par(mar = c(4,4,1,2)+0.1)
for(i in 9:(length(cols)-1)){
  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$holiday))){
    points(bike[bike$holiday==levels(bike$holiday)[j],i],
           bike[bike$holiday==levels(bike$holiday)[j],]$count, col = cbp[j], cex = 0.6)
  }

  plot(bike[,i], bike$count, xlab = cols[i], ylab = "count", cex.lab = 1.2, cex = 0.6)
  for(j in 1:length(levels(bike$workingday))){
    points(bike[bike$workingday==levels(bike$workingday)[j],i],
           bike[bike$workingday==levels(bike$workingday)[j],]$count, col = cbp[j], cex = 0.6)
```

```
  }
}
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend(x = "top", legend = levels(bike$holiday), cex = 0.7,
        col=cbp[1:length(levels(bike$holiday))], pch = 1, xpd = TRUE, bty = "n")
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend(x = "top", legend = levels(bike$workingday), cex = 0.7,
        col=cbp[1:length(levels(bike$workingday))], pch = 1, xpd = TRUE, bty = "n")
mtext("Interaction with holiday", side = 3, outer = TRUE, line = -2, cex = 1, at = 0.265)
mtext("Interaction with workingday", side = 3, outer = TRUE, line = -2, cex = 1, at = 0.775)
```

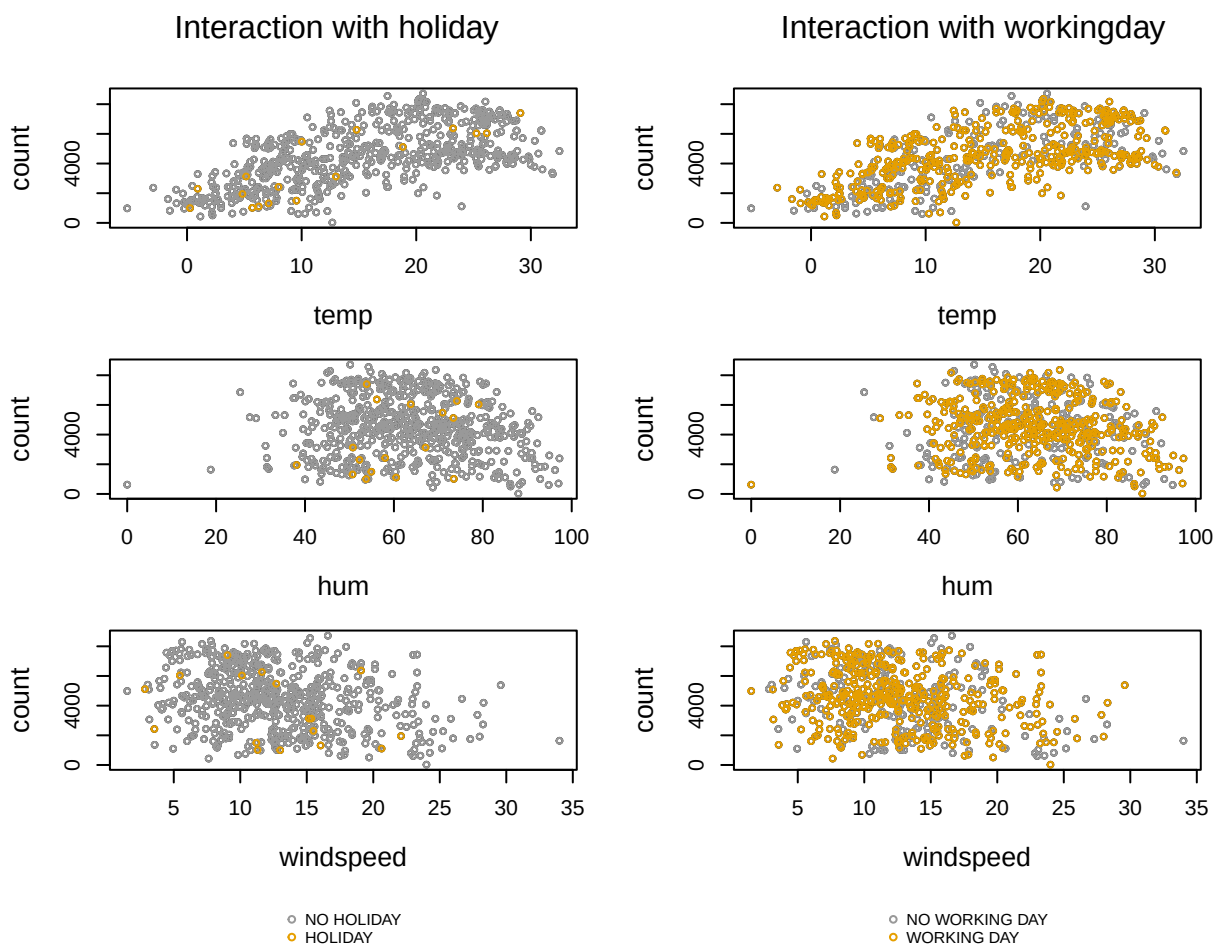

Figure 6: Day-type-dependent impact on count due to weather conditions

According to the scatter plots above, it seems that the relationships between each weather-related predictor and count are irrelevent to neither holiday nor working day. Hence, we conclude that the impact on count

due to weather conditions might be independent of day type.

1.3 Fit tree-based models and summarize their prediction performance. The tuning parameters for tree-based models should be determined via cross validation.

At this stage, three kinds of tree-based model is going to be fitted, including single decision tree, bagging (or random forest) and boosting. For each model, 5-fold cross-validation is used to evaluate the test MSE (mean squared error). For more details, cross validation is used to select the optimal cost-complexity parameter (α) for single decision tree, select the tuning parameter (m) for bagging (or random forest), and select the interaction depth (d), shrinkage parameter (λ) and number of trees (B) for boosting. The five folds used for cross-validation is randomly determined as follows.

```
k = 5
set.seed (10)
folds=sample(1:k, nrow(bike), replace=TRUE)
folds.n = table(folds)
```

According to the preliminary findings in EDA, temp, hum, wind speed, weather, month, year and season are considered as the important predictors. Hence, they are used in the following modeling procedure.

a. Single decision tree

Throughout the cross-validation procedure, the cost-complexity parameter α is searched among 2 to 12.

```
params = 2:12
cv.errors=matrix(NA, k, length(params))
for(i in 1:length(params)){
  param = params[i]
  for(j in 1:k){
      full.tree = tree(count~temp+hum+windspeed+weather+month+year+season, data = bike,
                  subset = folds != j, split = "deviance")
      fit = prune.tree(full.tree, best = param)
      pred = predict(fit, bike[folds == j,])
      print(summary(fit))
      cv.errors[j, i] = mean((bike[folds == j,]$count-pred)^2)
  }
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = expression(alpha), ylab = "test MSE")
abline(v = params[which.min(cv)], col = "red")
legend("topleft", c("test MSE", "CI"), lty = 1:2)
```

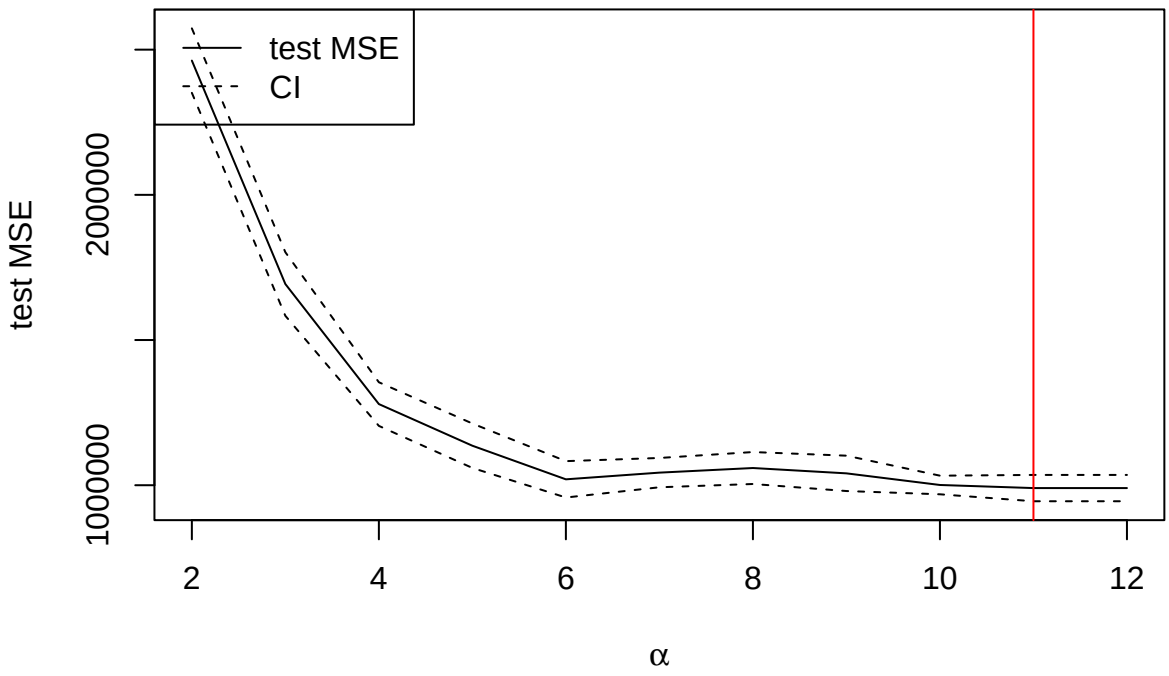

Figure 7: Cross validation for cost-complexity parameter

Figure 7 shows how the test MSE would vary as increasing the number of predictors under best subset selection procedure. The real line and the two dashed lines respectively refer to the mean, mean-SE and mean+SE for the test MSE under each number of predictors. It suggests that it might be better to set m at 11 for the lowest test MSE estimate. Similar figure will continuously be used to display the result of cross validation in the following analysis.

```
full.tree = tree(count~temp+hum+windspeed+weather+month+year+season, data = bike,
                 split = "deviance", minsize=5)
final.tree = prune.tree(full.tree, best = 11)
plot(final.tree, main = "decision tree")
text(final.tree, cex = 0.5)
```

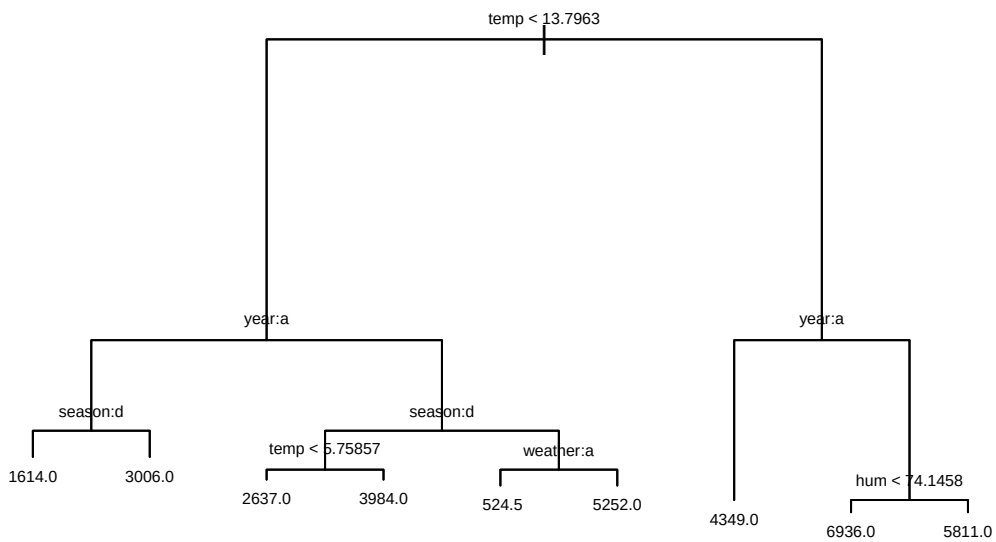

Figure 8: Single decision tree

Identify the important input variables and characterize the effects of weather conditions on the bike counts quantitatively

Figure 8 shows the graph for the fitted single tree. From top to down, one can see that temperature might be the most important predictor since it seems to reduce the most deviance at the first splitting. It seems that the overall average of count is smaller When the temperature is lower than 13.7963 than that when the temperature is higher than 13.7963.

For the case when temperature is lower than 13.7963, it also seems to be meaningful to consider temperature again, where the average of count might become smaller if the temperature is lower than 5.75857. Moreover, in this case, weather condition also seems to provide useful information, where it seems to tell that the average of count might become smaller when the weather is terrible such as rainy, snowy or storming. It might be also worth noticing that the temperature and weather condition seem to be especially useful in predicting count in 2012.

As for the case when temperature is greater than 13.7963, it seems that humidity might be an important predictor after temperature. From Figure 8, we can see that the count might become larger when humidity is lower than 74.1458. Moreover, the information provided by humidity seems to help more in 2012.

b. Random forest

Throughout the cross-validation procedure, the tuning parameter m is searched among 2 to 7.

```
params = 2:7
cv.errors=matrix(NA, k, length(params))
for(i in 1:length(params)){
```

```
  param = params[i]
  for(j in 1:k){
      fit = randomForest(count~temp+hum+windspeed+weather+month+year+season,
                                data = bike, subset = folds != j, split = "deviance",
                                mtry = param)
      pred = predict(fit, bike[folds == j,])
      cv.errors[j, i] = mean((bike[folds == j,]$count-pred)^2)
  }
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "m", ylab = "test MSE")
abline(v = params[which.min(cv)], col = "red")
legend("topleft", c("test MSE", "CI"), lty = 1:2)
```

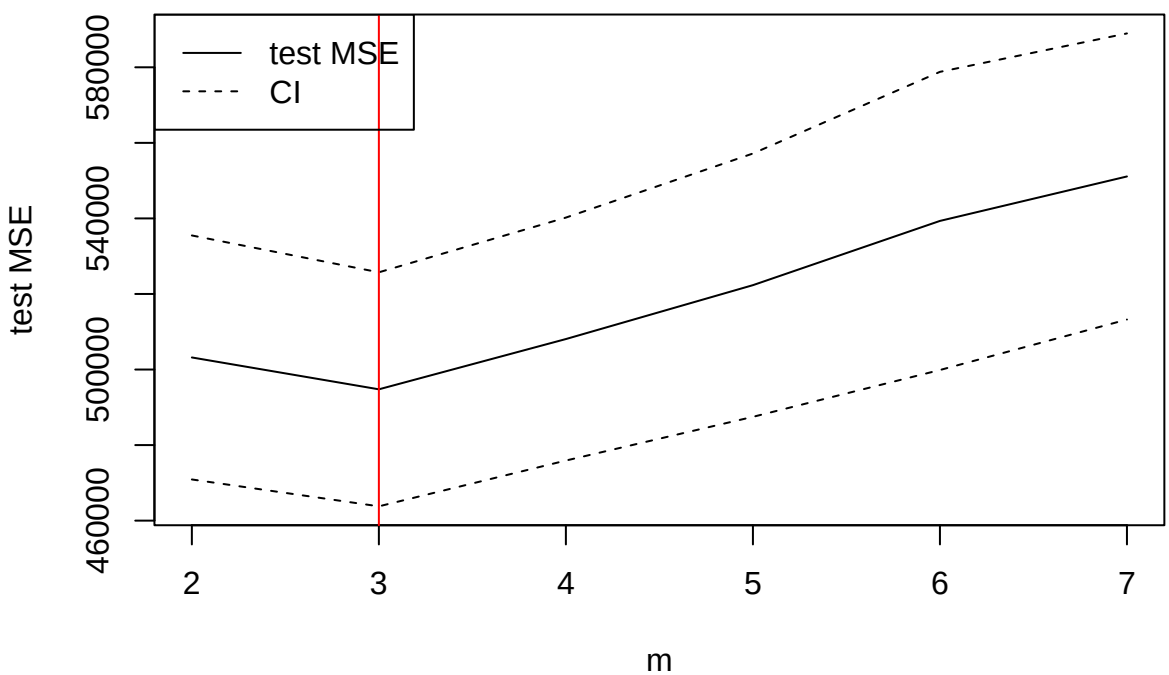

Figure 9: Cross validation for tuning parameter (m)

Figure 9 suggests that it might be better to set m at 3 for the lowest test MSE estimate. This result also implies that bagging, which use m equal to the number of all predictors, might not be a good choice. We thus fit again the random forest under m = 3 using the whole training data.

```
rf = randomForest(count~temp+hum+windspeed+weather+month+year+season,
                                data = bike, subset = folds != j, split = "deviance",
                                mtry = param)
plot(rf, main = "")
```

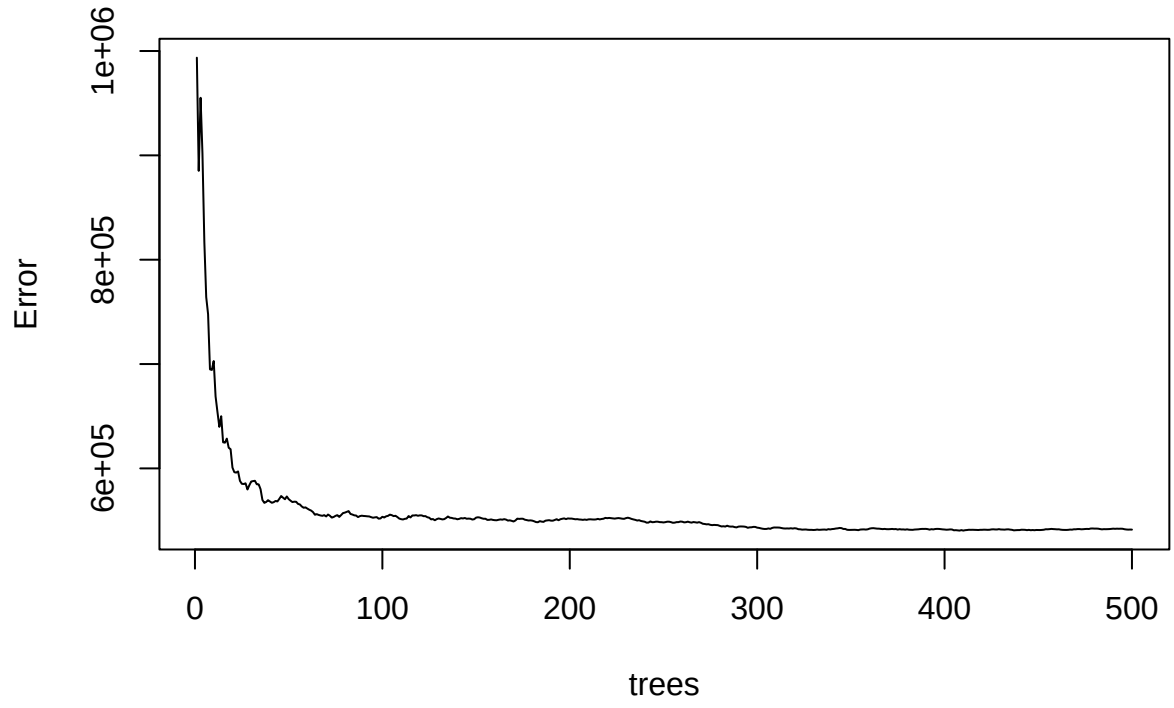

Figure 10: OOB MSE against number of trees

Figure 10 shows that 400 trees seems to be good enough. Hence, we refit the random forest under ntree = 400.

```
final.rf = randomForest(count~temp+hum+windspeed+weather+month+year+season,
                                data = bike, subset = folds != j, split = "deviance",
                                mtry = param, ntree = 400, importance = TRUE)
varImpPlot(final.rf, main = "")
```

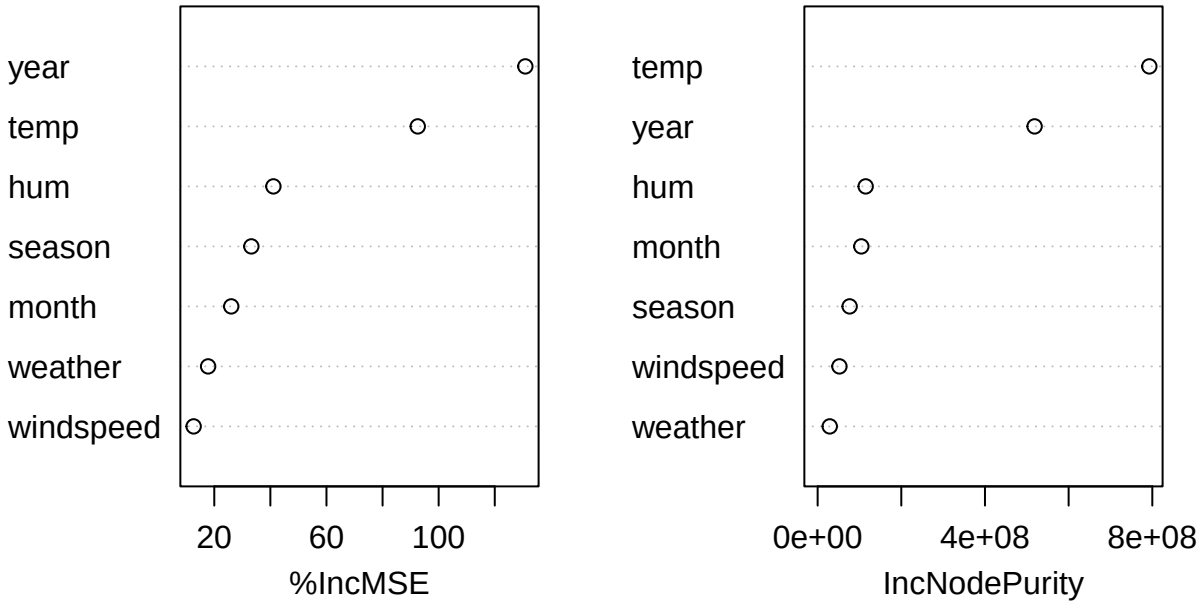

Figure 11: Importance plot for predictors

Identify the important input variables and characterize the effects of weather conditions on the bike counts quantitatively

Figure 11 shows the importance of those predictors used in the random forest. From top to down, one might conclude that year and temperature seem to be relatively more important predictors than humidity, season, month, weather and wind speed for that the former would seem to reduce more error in prediction of count.

c. Boosting

Throughout the cross-validation procedure, the interaction depth (d) is searched among 1 to 5, while the shrinkage parameter (λ) is searched among 0 to 0.02 in interval of 0.002. After d and λ are decided, the number of tree (B) would be tuned afterwards by conditioning on the optimal d and λ.

```
btcv = function(ntree){
    cv.errors = array(NA, dim = c(k, length(params1), length(params2)))
    for(i in 1:length(params1)){
      param1 = params1[i]
      for(l in 1:length(params2)){
        print(c(i, l))
        param2 = params2[l]
         for(j in 1:k){
          fit = gbm(count~temp+hum+windspeed+weather+month+year+season,
                                  data=bike[folds != j,],
                                  distribution= "gaussian", n.trees = ntree,
                                  interaction.depth=param1, shrinkage = param2,
```

```
                                    verbose = FALSE)
          pred = predict(fit, bike[folds == j,])
          cv.errors[j, i, l] = mean((bike[folds == j,]$count-pred)^2)
        }
      }
    }
    return(cv.errors)
}

params1 = 1:5 # d
params2 = seq(0, 0.02, 0.002) # lambda
if("bscv.error1.RData" %in% list.files()){
  source("bscv.error1.RData")
}else{
  cv.errors = btcv(5000)
  dump("cv.errors", "bscv.error1.RData")
}
cv = apply(cv.errors*array(matrix(rep(folds.n, length(params1)), nrow = k),
                           dim = c(k, length(params1), length(params2))),
                           c(2, 3), sum)/nrow(bike)
rownames(cv) = params1; colnames(cv) = params2
pheatmap(cv, cluster_rows = FALSE, cluster_cols = FALSE, main = "test MSE")
```

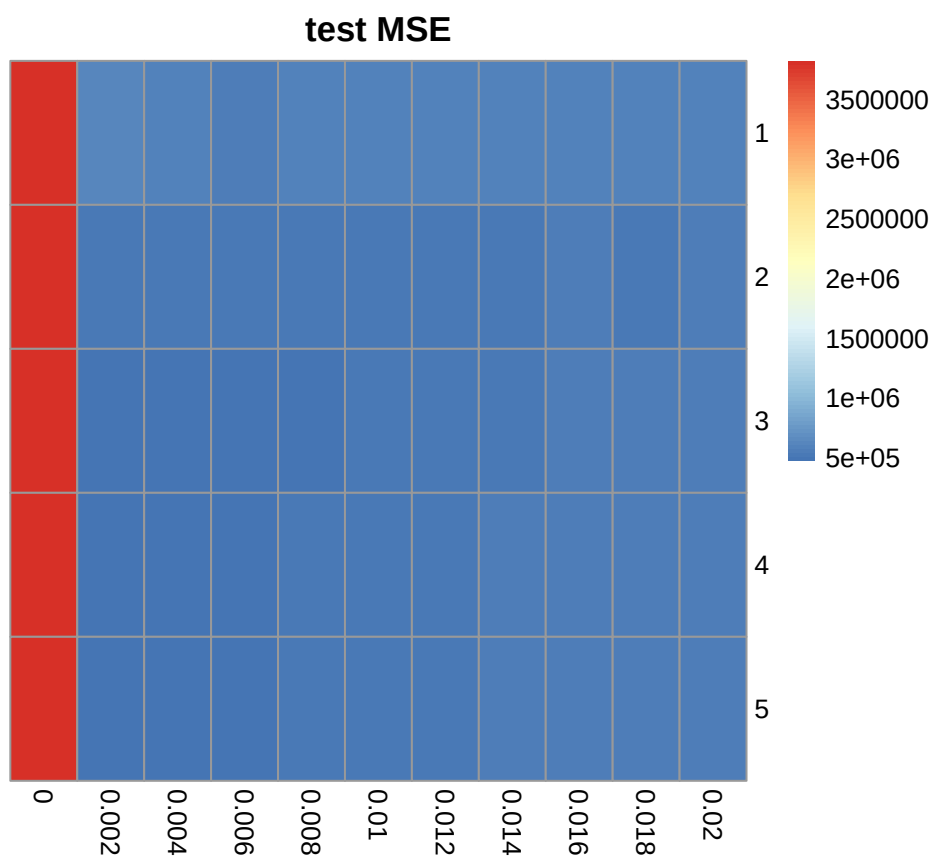

Figure 12: Cross validation for depth and shrinkage parameter

```
opt.params = c(params1[ifelse(which.max(cv) %% 4 == 0, 4, which.min(cv) %% 4)],
            params2[ifelse(which.min(cv) %% 4 == 0, which.min(cv) %/% 4,
                           which.min(cv) %/% 4 + 1)])
```

Figure 12 displays the test MSE estimates obtained under each setting (pair) of the parameters d and λ. Each row of the matrix shown refers to one distinct value of d, and each of the column refers to one distinct value of λ. It turns out that d = 2 and $\lambda = 0.004$ seem to provide the lowest test MSE.

```
params = seq(250, 5000, 250)
if("ntree.cv1.RData" %in% list.files()){
  source("ntree.cv1.RData")
}else{
  cv.errors=matrix(NA, k, length(params))
  for(i in 1:length(params)){
    print(params[i])
    for(j in 1:k){
          fit = gbm(count~temp+hum+windspeed+weather+month+year+season,
                                  data=bike[folds != j,],
                                  distribution= "gaussian", n.trees = params[i],
                                  interaction.depth=opt.params[1], shrinkage = opt.params[2],
                                  verbose = FALSE)
          pred = predict(fit, bike[folds == j,])
          cv.errors[j, i] = mean((bike[folds == j,]$count-pred)^2)
        }
  }
  dump("cv.errors", "ntree.cv1.RData")
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "trees", ylab = "test MSE")
opt.ntree = params[which.min(cv)]
abline(v = opt.ntree, col = "red")
legend("topleft", c("test MSE", "CI"), lty = 1:2)
```

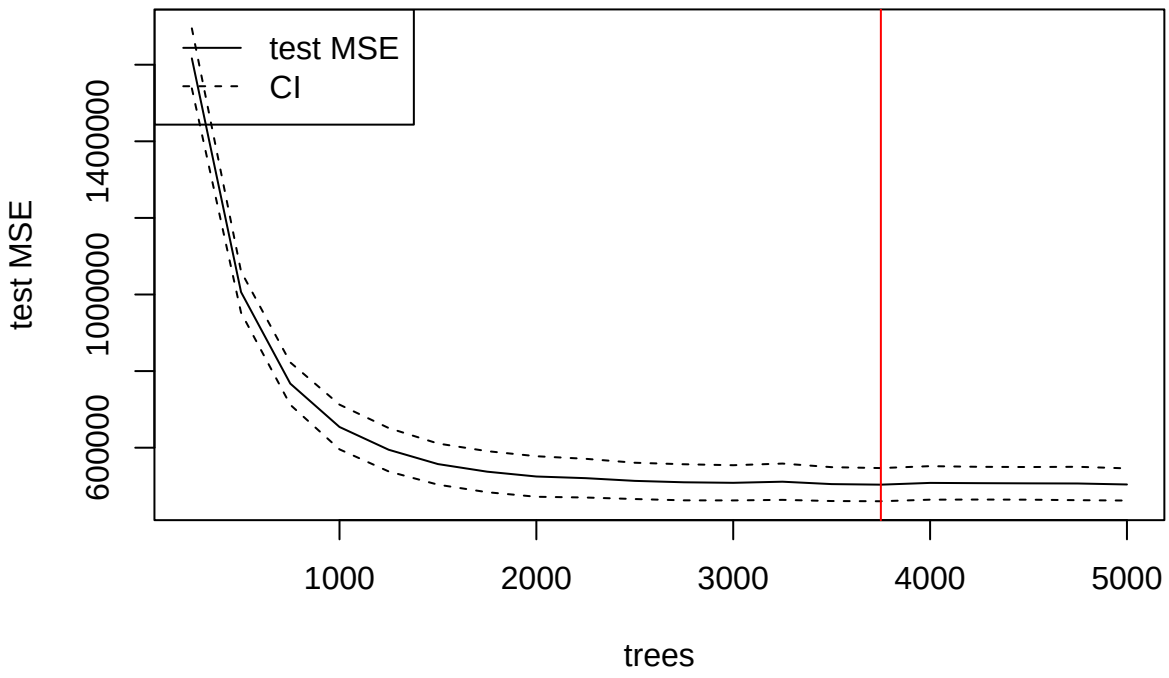

Figure 13: Cross validation for the number of trees

Figure 13 suggests that it might be better to set B at 3750 for the lowest test MSE. Finally, we fit again the boosting model by such optimal setting (including d, λ and B) using the whole training data.

```
boost = gbm(count~temp+hum+windspeed+weather+month+year+season,
                              data = bike, distribution = "gaussian",n.trees = opt.ntree,
                              interaction.depth = opt.params[1], shrinkage = opt.params[2])
imp = summary(boost, plotit = FALSE)
imp = imp[order(imp$rel.inf),]
barplot(imp$rel.inf, horiz = TRUE, names.arg = imp$var, las = 2,
        col = cbp[6], cex.names = 0.7, xlab = "relative influence")
```

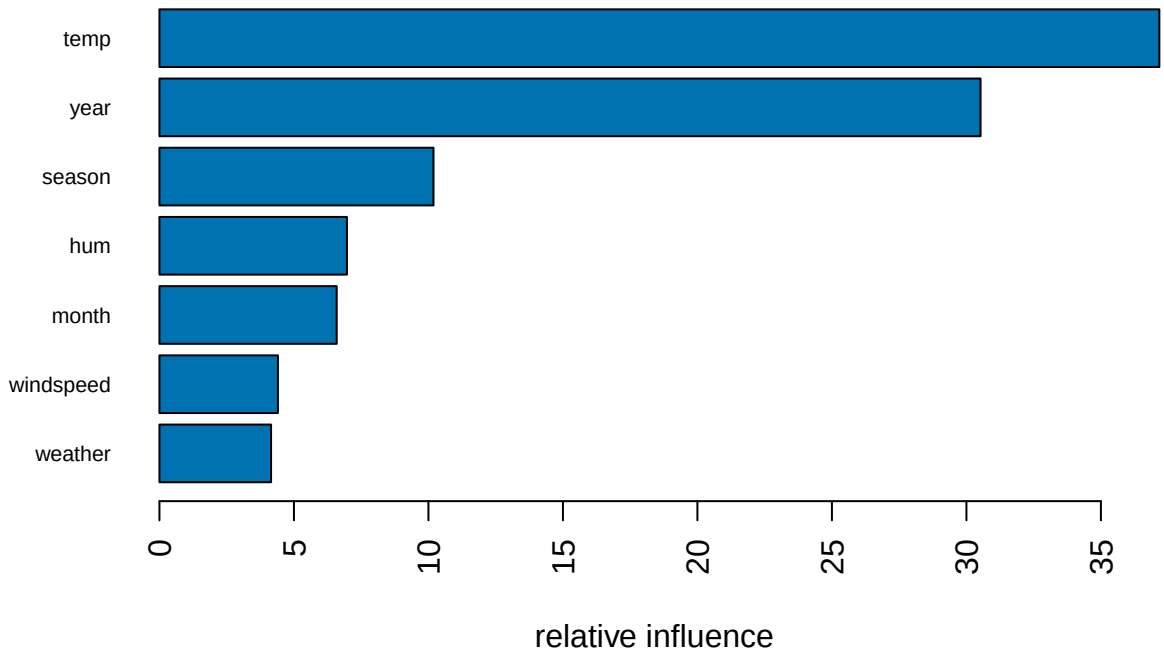

Figure 14: Importance plot for predictors

Identify the important input variables and characterize the effects of weather conditions on the bike counts quantitatively

Figure 14 shows the importance of predictors used in the boosting. From top to down, one might conclude that year and temperature seem to be relatively more important predictors than humidity, season, month, weather and wind speed for that the former would seem to reduce more error in prediction of count.

1.4 Fit a linear regression and a nonlinear regression model for the target variable count. Again, your models should go through a proper model selection procedure. Summarize the selected models with their prediction performance.

According to the preliminary findings in EDA, temp, hum, wind speed, weather, month, year, season, temp^2, temp×month and temp×season are considered as the candidates of important predictors.

a. Linear regression

Consider the model:

$$count = \beta_0 + \beta_1\, temp + \beta_2\, hum + \beta_3\, windspeed + \beta_4\, weather + \beta_5\, month + \beta_6\, year + \beta_7\, season + \epsilon$$

Since it is required to conduct some variable selection algorithm for more convincing result, Best subset selection under the framework of 5-fold cross validation is performed for selecting number of predictors p. Throughout the procedure, p is searched among 1 to 7.

```r
cv.errors=matrix(NA, k, 7)
for(j in 1:k){
  best.fit = regsubsets(count~temp+hum+windspeed+weather+month+year+season,
                        data = bike, subset = folds != j)
  for(i in 1:7){
    pred = predict.regsubsets(best.fit, bike[folds == j,], id = i)
    cv.errors[j, i] = mean((bike[folds == j,]$count-pred)^2)
  }
}
cv = apply(cv.errors*matrix(rep(folds.n, 7), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(1:7, cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "p", ylab = "test MSE")
opt.p = (1:7)[which.min(cv)]
abline(v = opt.p, col = "red")
legend("topleft", c("test MSE", "CI"), lty = 1:2)
```

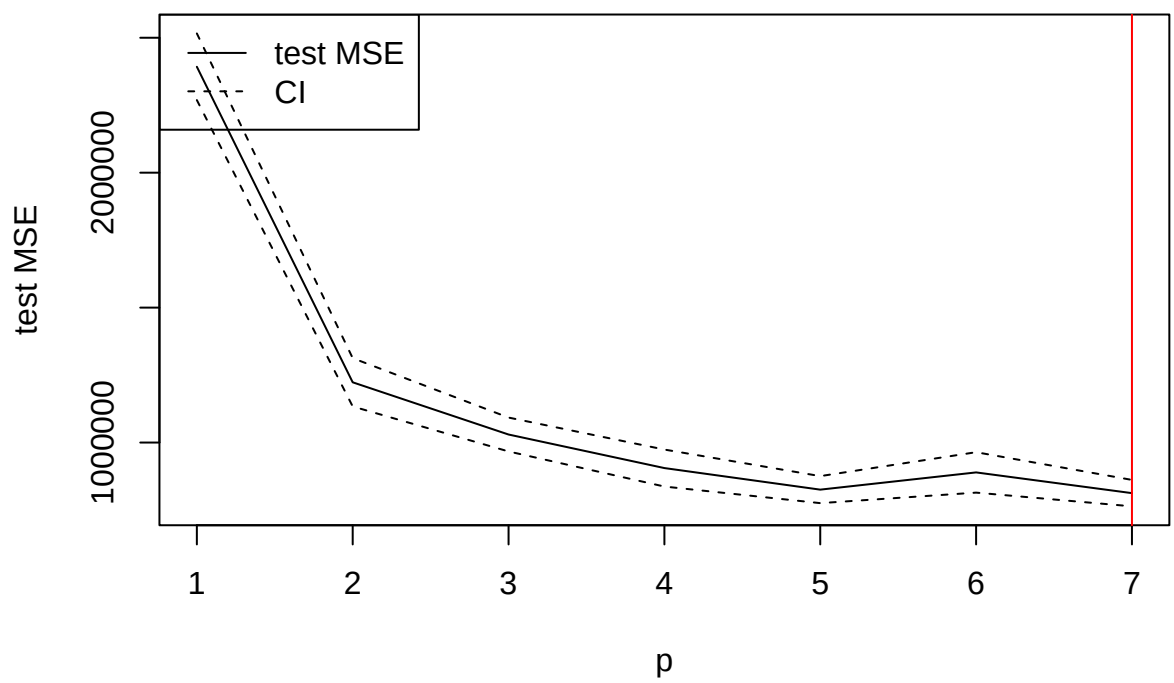

Figure 15: Cross validation with best subset selection

According to the mean of the test MSE (i.e., the real line), it is shown that 7 predictors (indicated by the red line) might provide the lowest test MSE. In other words, the full model under consideration might be the best one.

```
reg2 = lm(count~temp+hum+windspeed+weather+month+year+season, data = bike)
summ.kable(summary(reg2)$coef, "Summary for linear regression")
```

Table 6: Summary for linear regression

	Estimate	SE	t-value	p-value
(Intercept)	1579.70446	434.231547	3.6379311	0.0003000
temp	106.33558	10.096248	10.5321877	0.0000000
hum	-17.46277	3.334379	-5.2371886	0.0000002
windspeed	-48.22521	7.025732	-6.8640838	0.0000000
weatherMISTY	1559.01042	207.759799	7.5039080	0.0000000
weatherGOOD	1945.41338	221.919781	8.7662910	0.0000000
monthFEB	139.41792	170.724959	0.8166230	0.4144887
monthMAR	570.68550	186.764535	3.0556417	0.0023520
monthAPR	420.38326	283.432361	1.4831872	0.1385832
monthMAY	687.70528	306.502383	2.2437192	0.0252377
monthJUN	290.24406	328.870246	0.8825489	0.3778561
monthJUL	-582.57385	374.369243	-1.5561477	0.1202339
monthAUG	-161.04080	361.415302	-0.4455838	0.6560689
monthSEP	484.62166	317.442837	1.5266423	0.1274106
monthOCT	171.77735	283.146884	0.6066722	0.5443126
monthNOV	-455.58164	272.575132	-1.6713984	0.0951977
monthDEC	-249.83897	214.575789	-1.1643390	0.2447789
year2012	2001.55662	68.472747	29.2314345	0.0000000
seasonSUMMER	357.71323	240.286402	1.4886953	0.1371266
seasonFALL	1035.04266	257.517956	4.0193029	0.0000663
seasonWINTER	-794.65294	209.627139	-3.7907923	0.0001663

Identify the important input variables

Table 6 shows the summary for the linear regression. All the predictors are significant at 0.05 significance level, implying that each predictor seems to be important at least in sense of statistically significance. However, one may notice that the signs of the coefficients seem to be inconsistent with the findings in EDA. For example, the parameter that models the difference in count between January and July should be positive, but Table 6 shows negative estimate for it. Such issue might arise due to the fact that the current model is too simple.

b. Non-linear regression

Consider the model:

$$\begin{aligned} count =& \beta_0 + \beta_1\, temp + \beta_2\, hum + \beta_3\, windspeed + \beta_4\, weather + \beta_5\, month + \beta_6\, year + \beta_7\, season \\ &+ \beta_8\, temp^2 + \beta_9\, temp \times month + \beta_{10}\, temp \times season + \epsilon \end{aligned}$$

```
reg1 = lm(count~temp+hum+windspeed+weather+month+year+season
         +I(temp^2)+temp:month+temp:season, data = bike)
summ = summary(reg1)$coef
rownames(summ)[22] = c("$\\text{temp}^2$")
summ.kable(summ, "Summary for model nonlinear regression")
```

Table 7 shows the fitted result of the nonlinear regression model. As one can see, the quadratic term of temperature and the interaction term of temperature and season seem to be non-significant. The possible

Table 7: Summary for model nonlinear regression

	Estimate	SE	t-value	p-value
(Intercept)	1747.721317	557.962622	3.1323269	0.0018267
temp	110.939272	49.066668	2.2609905	0.0241504
hum	-22.050256	3.002825	-7.3431698	0.0000000
windspeed	-49.524510	6.282851	-7.8824899	0.0000000
weatherMISTY	1572.561415	184.296413	8.5327836	0.0000000
weatherGOOD	1941.875540	197.489335	9.8328122	0.0000000
monthFEB	172.948686	260.971533	0.6627109	0.5077938
monthMAR	-446.705645	354.686283	-1.2594387	0.2084071
monthAPR	-466.350220	687.934960	-0.6778987	0.4981215
monthMAY	1347.400520	1060.536174	1.2704899	0.2044483
monthJUN	4010.246807	1638.447541	2.4475894	0.0146937
monthJUL	7649.420358	2491.400746	3.0703292	0.0022440
monthAUG	-1588.237516	2457.910661	-0.6461738	0.5184368
monthSEP	1786.837492	1736.895362	1.0287537	0.3040485
monthOCT	-1908.321832	643.074814	-2.9674958	0.0031332
monthNOV	-2040.922708	555.981572	-3.6708460	0.0002654
monthDEC	-1418.634916	371.128559	-3.8224892	0.0001472
year2012	1972.773647	62.764022	31.4316001	0.0000000
seasonSUMMER	1590.245928	1622.583838	0.9800701	0.3274832
seasonFALL	1786.575854	515.696536	3.4643937	0.0005729
seasonWINTER	-655.449238	448.298549	-1.4620820	0.1442909
temp^2	-2.175562	1.960894	-1.1094745	0.2677113
temp:monthFEB	-6.286694	41.278069	-0.1523011	0.8790054
temp:monthMAR	117.060656	43.610833	2.6842105	0.0074897
temp:monthAPR	101.131890	64.688005	1.5633793	0.1185396
temp:monthMAY	15.419681	80.837486	0.1907491	0.8487927
temp:monthJUN	-99.600999	101.204480	-0.9841560	0.3254723
temp:monthJUL	-228.591295	127.172113	-1.7974954	0.0728064
temp:monthAUG	118.131244	124.955374	0.9453875	0.3448770
temp:monthSEP	-9.932203	98.957893	-0.1003680	0.9200888
temp:monthOCT	194.396294	66.019827	2.9445138	0.0033716
temp:monthNOV	184.973729	63.516568	2.9122123	0.0037347
temp:monthDEC	162.401749	53.602806	3.0297248	0.0025630
temp:seasonSUMMER	-53.675302	66.174813	-0.8111138	0.4176518
temp:seasonFALL	-61.952424	36.609220	-1.6922629	0.0911633
temp:seasonWINTER	11.006586	37.290023	0.2951617	0.7679820

reason might be that month, season and temperature are highly associated (as shown in EDA). Hence, three models were tried as follows.

Non-linear regression (revised version 1)

The first revised version is removing the interaction term of temperature and month.

```
reg12 = update(reg1, .~.-temp:month)
summ = summary(reg12)$coef
rownames(summ)[22] = c("$\\text{temp}^2$")
summ.kable(summ, "Summary for nonlinear regression (revised 1)")
```

Table 8 shows the fitted result of the nonlinear regression model (revised version 1). As one can see, all the

Table 8: Summary for nonlinear regression (revised 1)

	Estimate	SE	t-value	p-value
(Intercept)	1039.100220	498.795629	2.0832184	0.0376843
temp	250.423296	39.934916	6.2707857	0.0000000
hum	-20.658455	3.101606	-6.6605662	0.0000000
windspeed	-51.122403	6.532291	-7.8261062	0.0000000
weatherMISTY	1587.524181	192.161109	8.2614229	0.0000000
weatherGOOD	1989.285328	205.798777	9.6661669	0.0000000
monthFEB	-56.928858	163.334866	-0.3485408	0.7275651
monthMAR	328.642089	194.976609	1.6855462	0.0924403
monthAPR	49.097360	272.244039	0.1803432	0.8569484
monthMAY	300.752717	289.094972	1.0403250	0.2986379
monthJUN	36.033290	308.357421	0.1168556	0.9070164
monthJUL	-125.675510	349.613386	-0.3594700	0.7193791
monthAUG	-247.957818	334.878056	-0.7404421	0.4593421
monthSEP	-394.936524	312.486653	-1.2638509	0.2068094
monthOCT	-487.424740	269.831730	-1.8064026	0.0713922
monthNOV	-874.172513	261.504532	-3.3428580	0.0008846
monthDEC	-521.172935	207.424976	-2.5125852	0.0122649
year2012	1968.824504	64.024053	30.7513256	0.0000000
seasonSUMMER	4451.320256	830.154956	5.3620354	0.0000001
seasonFALL	1157.412133	403.478759	2.8685826	0.0042786
seasonWINTER	-268.191618	375.439330	-0.7143408	0.4753141
$temp^2$	-4.183932	1.136041	-3.6829075	0.0002529
temp:seasonSUMMER	-165.978740	33.921976	-4.8929562	0.0000013
temp:seasonFALL	11.719161	22.961368	0.5103860	0.6099822
temp:seasonWINTER	-43.108092	29.414246	-1.4655515	0.1433317

predictors are significant at significance level 0.05.

Non-linear regression (revised version 2)

The second revised version is removing both the quadratic term of temperature and the interaction term of temperature and season.

```
reg13 = update(reg1, .~.-temp:season-I(temp^2))
summ.kable(summary(reg13)$coef, "Summary for nonlinear regression (revised 2)")
```

Table 9 shows the fitted result of the nonlinear regression model (revised version 2). As one can see, all the predictors are significant at significance level 0.05.

Non-linear regression (revised version 3)

The third revised version is removing both the quadratic term of temperature and the interaction term of temperature and season. Moreover, consider the linear spline in terms of temperature. The model is rewritten as follows.

$$\begin{aligned} count =& \beta_0 + \beta_1\, temp + \beta_2\, hum + \beta_3\, windspeed + \beta_4\, weather + \beta_5\, month + \beta_6\, year + \beta_7\, season \\ &+ \beta_8\, (temp - \xi)_+ + \epsilon \end{aligned}$$

where ξ denote the position of knot in temperature and $(temp - \xi)_+$ means positive part.

Table 9: Summary for nonlinear regression (revised 2)

	Estimate	SE	t-value	p-value
(Intercept)	1874.63434	391.357244	4.7900847	0.0000021
temp	104.48861	25.943073	4.0276112	0.0000642
hum	-22.33027	3.011083	-7.4160261	0.0000000
windspeed	-50.38034	6.299016	-7.9981287	0.0000000
weatherMISTY	1565.28001	184.837217	8.4684245	0.0000000
weatherGOOD	1936.11242	197.734985	9.7914510	0.0000000
monthFEB	224.61838	257.430063	0.8725414	0.3832918
monthMAR	-268.06318	267.902943	-1.0005981	0.3174588
monthAPR	-126.59547	408.686371	-0.3097619	0.7568587
monthMAY	2082.36294	655.036224	3.1790043	0.0015605
monthJUN	5671.91910	1077.205910	5.2653992	0.0000002
monthJUL	10239.25769	1246.166179	8.2166070	0.0000000
monthAUG	696.20071	1441.961295	0.4828151	0.6294181
monthSEP	3343.51742	758.173191	4.4099652	0.0000124
monthOCT	-779.29309	435.364277	-1.7899794	0.0740042
monthNOV	-1103.33161	410.874265	-2.6853267	0.0074634
monthDEC	-943.17267	307.788055	-3.0643576	0.0022877
year2012	1969.35557	62.744340	31.3869833	0.0000000
seasonSUMMER	600.44340	218.234109	2.7513729	0.0061292
seasonFALL	949.70899	234.584904	4.0484659	0.0000589
seasonWINTER	-744.20722	185.862759	-4.0040685	0.0000708
temp:monthFEB	-15.33100	40.618998	-0.3774342	0.7059958
temp:monthMAR	82.49897	31.760575	2.5975277	0.0096399
temp:monthAPR	43.77827	34.644899	1.2636282	0.2068959
temp:monthMAY	-63.99831	39.904186	-1.6037993	0.1093292
temp:monthJUN	-223.29668	50.766294	-4.3985224	0.0000131
temp:monthJUL	-397.60037	51.312537	-7.7486009	0.0000000
temp:monthAUG	-39.19579	61.637016	-0.6359132	0.5250961
temp:monthSEP	-138.43247	43.172299	-3.2065114	0.0014212
temp:monthOCT	78.44227	35.069942	2.2367380	0.0257013
temp:monthNOV	86.43556	43.065354	2.0070788	0.0452274
temp:monthDEC	109.30427	44.459663	2.4585041	0.0142571

To determine the optimal location of knot for the linear spline term, cross-validation similar as previous, is conducted. According to the finding in EDA, it is presumed that the optimal location should be somewhere near 25 with respect to temperature. Hence, the knot location is searched among 20 to 30.

```
params = 20:30
cv.errors=matrix(NA, k, length(params))
for(i in 1:length(params)){
  param = params[i]
  for(j in 1:k){
      fit = lm(count~hum+windspeed+weather+month+year+season
               +bs(temp, degree = 1, knots = c(param)), data = bike, subset = folds != j)
      pred = predict(fit, bike[folds == j,])
      print(summary(fit))
      cv.errors[j, i] = mean((bike[folds == j,]$count-pred)^2)
  }
}
```

```
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "knot location", ylab = "test MSE")
abline(v = params[which.min(cv)], col = "red")
legend("topleft", c("test MSE", "CI"), lty = 1:2)
```

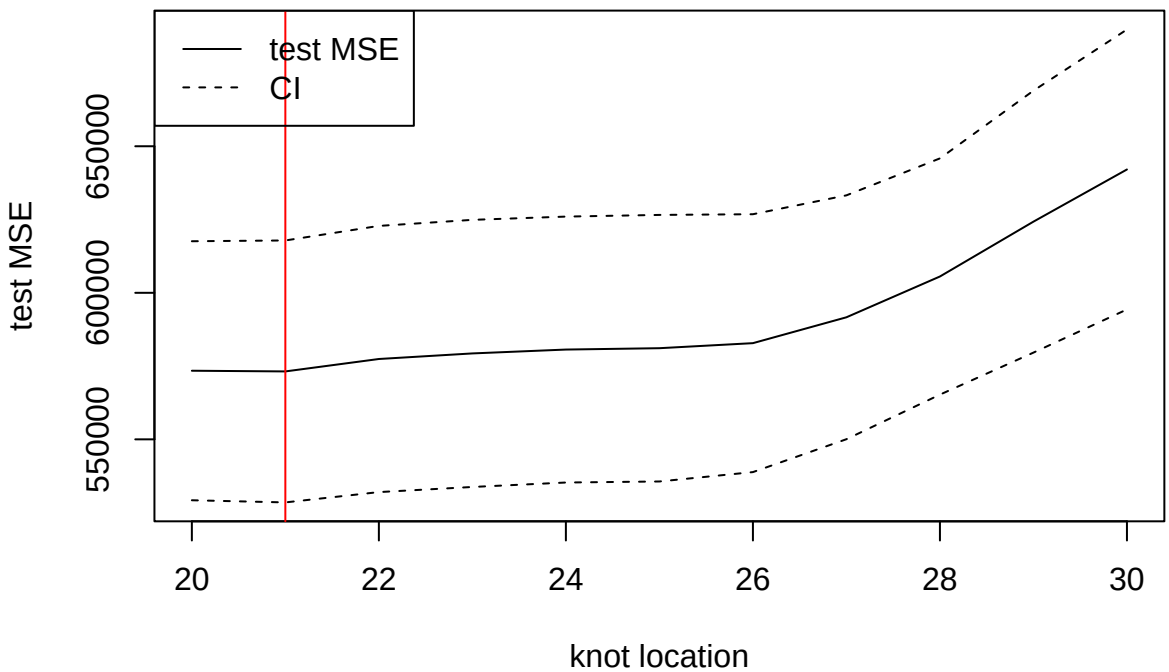

Figure 16: Cross validation for the position of knot

Figure 16 suggests that the knot location might be better to be set at 21 for the lowest test MSE estimate.

```
reg14 = lm(count~hum+windspeed+weather+month+year+season
         +bs(temp, degree = 1, knot = c(21)), data = bike)
summ = summary(reg14)$coef
rownames(summ)[21:22] = c("temp", "$(\\text{temp}-\\xi)_{+}$")
summ.kable(summ, "Summary for nonlinear regression (revised 3)")
```

Table 10 shows the fitted result of the nonlinear regression model (revised version 3). As one can see, all the predictors are significant at significance level 0.05 though the overall significance of month seems to decline a little bit than before. Though one might suspect whether the positive part of temperature can be regarded as significant, here we treat it as an important predictor since its p-values is actually very closed to 0.05.

Table 10: Summary for nonlinear regression (revised 3)

	Estimate	SE	t-value	p-value
(Intercept)	920.87034	393.288613	2.3414620	0.0195562
hum	-21.80851	3.057284	-7.1332937	0.0000000
windspeed	-52.45247	6.398851	-8.1971701	0.0000000
weatherMISTY	1550.85440	188.877511	8.2109002	0.0000000
weatherGOOD	1925.69185	201.757057	9.5446072	0.0000000
monthFEB	-18.73455	155.880681	-0.1201852	0.9043793
monthMAR	209.85210	172.970099	1.2132276	0.2255516
monthAPR	-82.46668	261.747183	-0.3150624	0.7528310
monthMAY	139.93834	283.116514	0.4942783	0.6213025
monthJUN	97.59605	299.497785	0.3258657	0.7446471
monthJUL	-118.50214	342.981251	-0.3455062	0.7298429
monthAUG	-238.62980	328.642147	-0.7261083	0.4680741
monthSEP	-214.56751	295.597494	-0.7258773	0.4682156
monthOCT	-318.71525	261.295073	-1.2197522	0.2230695
monthNOV	-719.83820	248.977305	-2.8911800	0.0039861
monthDEC	-417.67987	195.676232	-2.1345458	0.0332294
year2012	1938.97811	62.511956	31.0177162	0.0000000
seasonSUMMER	631.49709	219.878284	2.8720303	0.0042321
seasonFALL	1026.20609	234.112957	4.3833802	0.0000139
seasonWINTER	-763.69137	190.594710	-4.0068865	0.0000698
temp	4132.92222	270.304936	15.2898511	0.0000000
$(\text{temp} - \xi)_+$	2526.52158	371.903035	6.7934955	0.0000000

1.5 Evaluate the prediction performance via testing error on testing set

For convenience, the linear regression is abbreviated as LR, and the non-linear regression of revised version from 1 to 3 are abbreviated as NLR 1, NLR 2 and NLR 3 in the following discussion.

```
test.summary = matrix(NA, nrow = 1, ncol = 7)
colnames(test.summary) = c("decision tree", "random forest", "boosting", "LR",
                           "NLR 1", "NLR 2", "NLR 3")
rownames(test.summary) = c("RMSE")
models = list(final.tree, final.rf, boost, reg2, reg12, reg13, reg14)
for(i in 1:7){
  fit = models[[i]]
  pred = as.data.frame(predict(fit, bike.test))
  test.summary[,i] = sqrt(mean(as.vector((bike.test$count-pred))[[1]]^2))
}
```

```
kable(as.data.frame(test.summary), booktabs = TRUE, linesep = "",
      align = "c", caption = "Summary of testing error for each model") %>%
  kable_styling(latex_options = c("hold_position"))
```

Table 11 shows the test root mean squared error (RMSE) of each model, suggesting that boosting might provide the lowest RMSE. Random forest and NLR 3 which consumed much less time to fit than boosting did also seem to be good for that their test RMSE do not seem to be larger than boosting's so much.

Table 11: Summary of testing error for each model

	decision tree	random forest	boosting	LR	NLR 1	NLR 2	NLR 3
RMSE	826.842	605.7467	582.7129	745.6627	640.297	629.1392	617.3278

1.6 Brief analysis summary

Several important findings and conclusion are summarized as follows.

1. Temperature and year seems to be the most important predictors in predicting the count of bike sharing use. The reason is because that these two are used for splitting very early in the fitted decision tree and seem to reduce the most deviance during the fitting of random forest and boosting. Moreover, the linear and non-linear regression analysis also suggest their importance in terms of significance according to the p-values.

2. Humidity also seem to provide useful information in prediction of the count since they are also used in splitting for the decision tree and are significant in the linear and non-linear regression. Furthermore, it might be also worth noting that humidity is listed in the very front place in random forest (rank 3) and boosting (rank 4).

3. Weather condition also seems to be an important predictor in that it is remained in the decision tree after pruning and is significant in linear and non-linear regression. It is also shown that weather condition might be especially useful in 2012.

4. The overall impact on count due to temperature is that the count increases as temperature climb from extremely low to high. As for the quadratic-like effect of temperature on count, which have been mentioned in EDA, we might need more information to confirm since in NLR 3 the spline term might be thought as not very significant and the sign of its coefficient is also not very consistent with the findings in EDA.

5. Overall, the impact on count due to weather does not seem stationary. Especially there seems to exist some seasonal effect which might interact with the impact of temperature on count (see the interaction terms displayed in the non-linear regression model). Simply speaking, increase in temperature seems to associate with more use of bike sharing in cold seasons (fall and winter) while associate with less use of bike sharing in hot seasons (spring and summer).

2 Problem 2: Airline Customer Satisfaction

This dataset contains an airline passenger satisfaction survey. The data collect the demographic information about the customers, their feedback regarding the flight experiences and some flight information.

The goal of the analysis has two folds:

- Find a good model for the classification of customer satisfaction, and summarize the performance.
- Identify the important aspects of the flight experience related to customer satisfaction (or dissatisfaction). Based on your findings, give some specific suggestions (provided with the data evidence) to the airline company for achieving a higher customer satisfaction.
- Add some noise factors (both continuous variable and categorical variable) in your data set (call these added variables z1-z?), and check the ranking of variable importance of these z-variables relative to the original meaningful input variables. Make your comments regarding this experiment.

More data descriptions can be found at https://www.kaggle.com/teejmahal20/airline-passenger-satisfaction

```
survey <- read.csv(file="airline.csv") #read data
head(survey)
```

```
##       id Gender     Customer.Type Age  Type.of.Travel    Class Flight.Distance
## 1  70172   Male    Loyal Customer  13 Personal Travel Eco Plus             460
## 2   5047   Male disloyal Customer  25 Business travel Business             235
## 3 110028 Female    Loyal Customer  26 Business travel Business            1142
## 4  24026 Female    Loyal Customer  25 Business travel Business             562
## 5 119299   Male    Loyal Customer  61 Business travel Business             214
## 6 111157 Female    Loyal Customer  26 Personal Travel      Eco            1180
##   Inflight.wifi.service Departure.Arrival.time.convenient
## 1                     3                                 4
## 2                     3                                 2
## 3                     2                                 2
## 4                     2                                 5
## 5                     3                                 3
## 6                     3                                 4
##   Ease.of.Online.booking Gate.location Food.and.drink Online.boarding
## 1                      3             1              5               3
## 2                      3             3              1               3
## 3                      2             2              5               5
## 4                      5             5              2               2
## 5                      3             3              4               5
## 6                      2             1              1               2
##   Seat.comfort Inflight.entertainment On.board.service Leg.room.service
## 1            5                     5                4                3
## 2            1                     1                1                5
## 3            5                     5                4                3
## 4            2                     2                2                5
## 5            5                     3                3                4
## 6            1                     1                3                4
##   Baggage.handling Checkin.service Inflight.service Cleanliness
## 1                4               4                5           5
## 2                3               1                4           1
## 3                4               4                4           5
```

```
## 4                3               1                4           2
## 5                4               3                3           3
## 6                4               4                4           1
##   Departure.Delay.in.Minutes Arrival.Delay.in.Minutes            satisfaction
## 1                         25                       18 neutral or dissatisfied
## 2                          1                        6 neutral or dissatisfied
## 3                          0                        0               satisfied
## 4                         11                        9 neutral or dissatisfied
## 5                          0                        0               satisfied
## 6                          0                        0 neutral or dissatisfied
```

```
survey1 <- na.omit(survey) #remove missing data
cols = colnames(survey1)
survey1$Class = factor(survey1$Class, levels = c("Eco", "Eco Plus", "Business"))
survey1$Gender = factor(survey1$Gender)
survey1$Customer.Type = factor(survey1$Customer.Type)
survey1$Type.of.Travel = factor(survey1$Type.of.Travel)
survey1$satisfaction = factor(survey1$satisfaction)
```

2.1 Train-test splitting

To evaluate the prediction performance of each fitted model throughout the analysis, the data is splitted into training set and testing set. The training set and the testing set respectively accounts for 80% (82875 customers) and 20% (20719 observations) of the original data. The splitting is determined by simple random sampling without replacement (with random seed = 10).

```
train = rep(FALSE, nrow(survey1))
set.seed(10)
train[sample(1:nrow(survey1), round(nrow(survey1) * 0.8))] = TRUE
test = !train
```

Notice that throughout the study, every procedure involved with variable selection (including EDA) or model fitting will be conducted only on the training set, and hence no information of testing set would be misused.

```
## Use information from the training set only!
survey1.test = survey1[test,]
survey1 = survey1[train, ]
```

2.2 Brief EDA

1. Data description

This data set consists of 103594 observations on 24 variables after removing the missing values. Gender, Customer.Type, Type.of.travel and satisfaction are categorical variables, and Age, Fight distance, Departure delay in minutes (DDIM) and Arrival delay in minutes (ADIM) are approximately continuous. The remaining variables except for id are regarding to the feedback from the customers, which might be suitable to be treated as quantitatively discrete variable in the following study. For convenience, those feedback variables will be called items through out the analysis. To achieve the first goal of this analysis, satisfaction is going to be defined as the response variable, and the other variables except for id are going to be defined as the predictors.

2. Summary for the variables

Since the number of variables is quite huge in this data set, it seems that only basic summary table can be shown for each variable at this stage. Due to the space consideration, the summary tables are displayed in Appendix A. Table 13, 14, 15, 16, 17, 18, 19, 20, 21, 22 show the counts of data among the levels of each discrete variable, including the categorical and the quantitatively continuous ones. Table 23 shows five-number-summary and means for the continuous variables.

According to those tables, several interesting findings are summarized as follows

1. There seems to be several leverage points in the data with respect to Gate.location, Seat.comfort and On.board.service. Those customers seemed to give extremely low remark toward those corresponding items. However, no any measure would be taken toward such issue in consideration that the sample size seems to be quite large enough so that model built on it might not be so sensitive to those special data points.
2. All the continuous variables seem to be severely right-skewed except for Age, which does not show obvious difference between its mean and median.
3. We have almost the same number of female and male in the data, implying that no gender bias would happen.
4. Most of the customers are loyal customers, implying the data might be biased in terms of loyalty and hence we might overestimate the satisfaction of customers based on this data. To deal with such issue, it might be necessary to include Customer type as an important predictor.
5. Most of the customers took the flight for business travel.
6. No significant difference on number of customers seem to appear between Eco and Business class.
7. The distribution of score given on each item seems to be left-skewed for most of the items except for Inflight.wifi.service, Ease.of.Online.booking and Gate.location.
8. The satisfied customers is less than those who feel neutral or dissatisfied. Considering that most of the customers are loyal, such proportion (43.3%) of satisfaction might not be regarded as a good news.

3. Univariate distribution and pair-wise relation among the variables

a. Univariate distribution for the continuous variables

```
par(mfrow = c(1, 4))
for(i in c(4, 7, 22, 23)){
  hist(survey1[,i], xlab = cols[i], main = "")
}
```

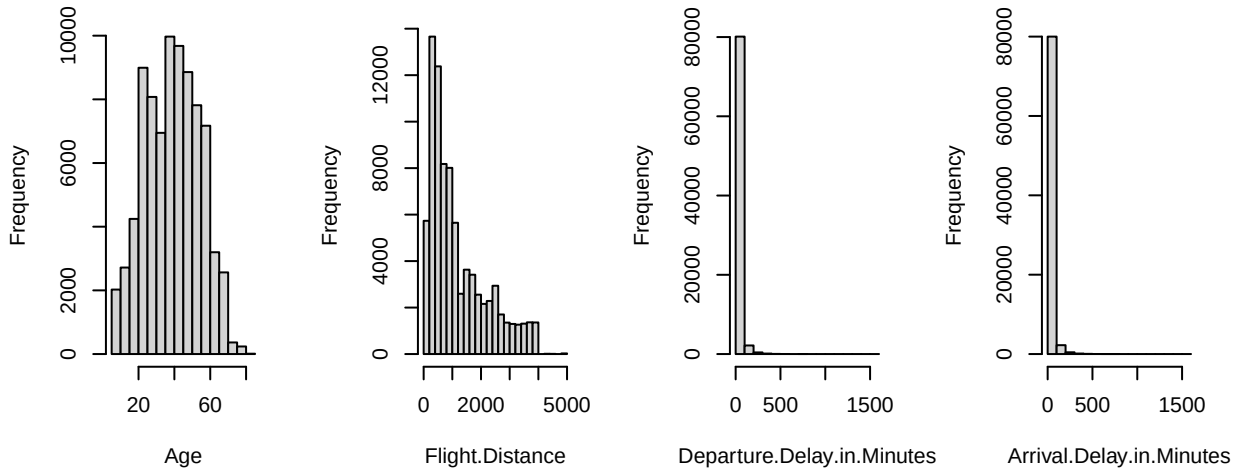

Figure 17: Histograms for the continuous variables

Figure 17 shows the univariate histograms for the continuous variables. We can see that they all look right-skewed except for Age, which distribute quite symmetrically. Overall, these histograms display consistent results with Table 23.

a. Univariate distribution for each item

```
par(mfrow = c(2, 7), mar = c(5, 2, 4, 2)+0.1)
for(i in 8:21){
  hist(survey1[,i], xlab = cols[i], main = "", cex.lab = 0.6)
}
```

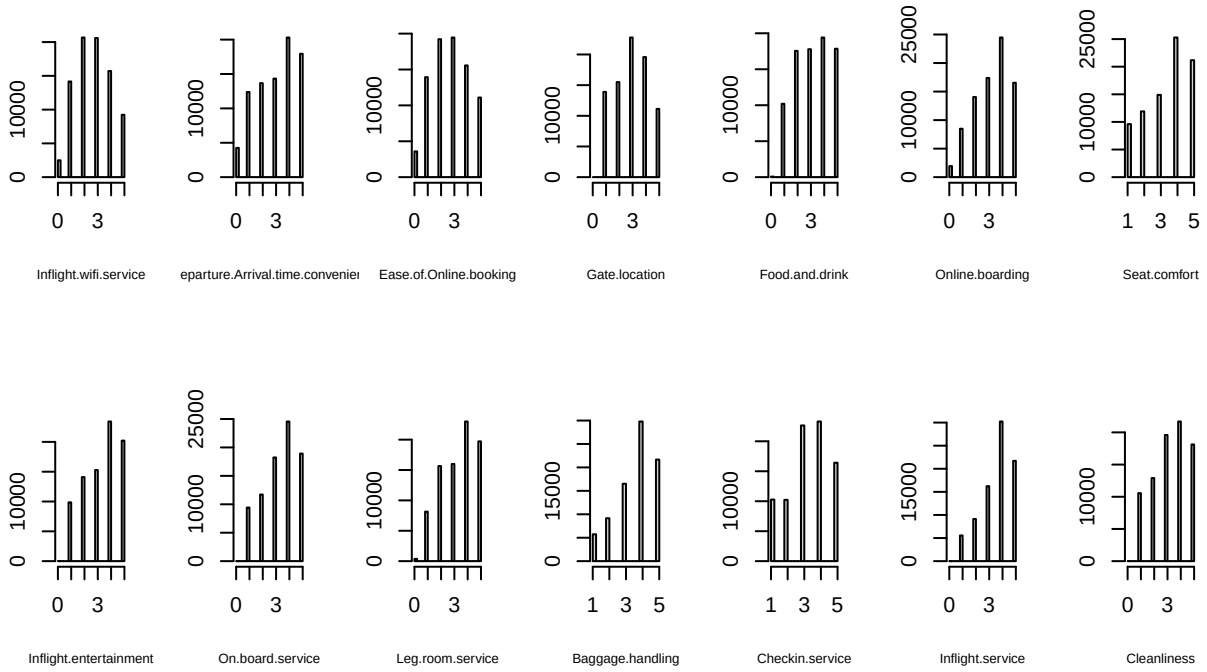

Figure 18: Histograms for the continuous variables

Figure 18 shows the univariate histograms for the items. We can see that most of them look left-skewed except for Inflight.wifi.service, Ease.of.Online.booking and Gate.location, which seem to distribute symmetrically. Overall, these histograms display consistent results with Table 15, 16, 17, 18, 19, 20, 21, 22.

c. Pairwise relation

Due to space consideration and the fact that the major purpose of this analysis is only to classify customer satisfaction, we will mainly focus the relation between each predictor and satisfaction in the following. However, due to the psychological nature of this survey data, it might also interesting first to examine the correlation among the item scores in hope of finding interesting psychological construct for further analysis.

Correlation among item scores

```
corrplot(cor(survey1[8:21]), cl.cex = 0.5, tl.cex = 0.5)
```

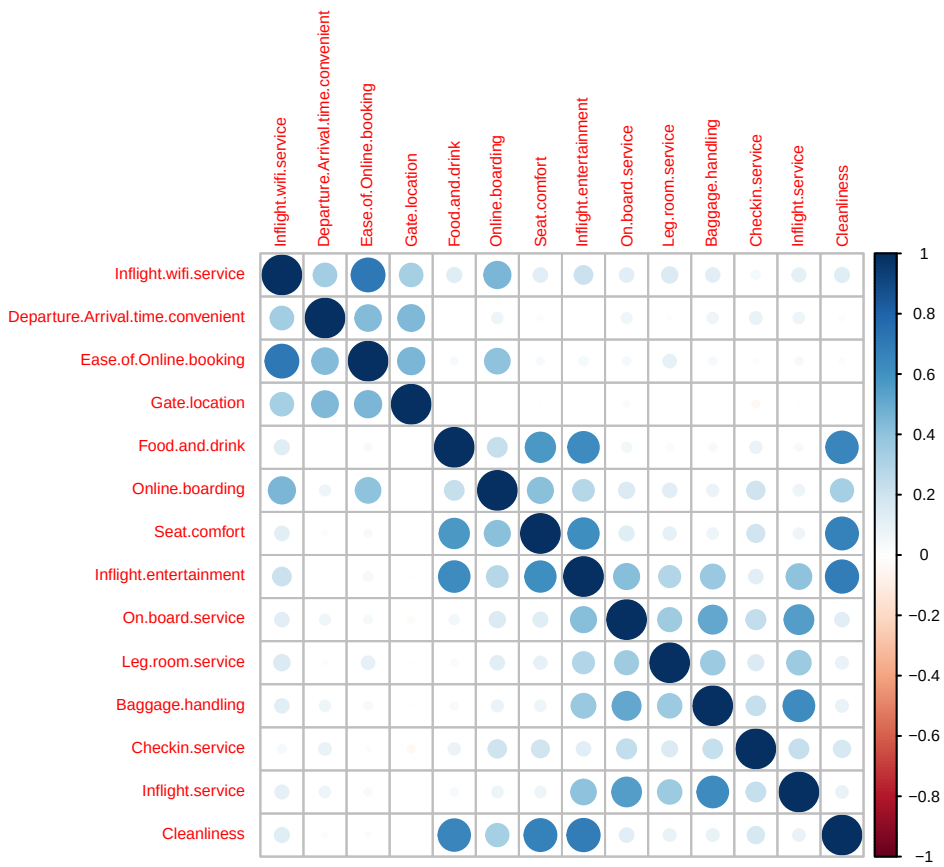

Figure 19: Corrlation matrix for the 14 items

Figure 19 shows the correlation matrix for the 14 items. One can see that there seems to be three blocks on the diagonal, in each of which the items seem to have relatively high correlation among others. Such facts suggests that some of the items might carry similar information, and hence one may only need a subset of them in classifying customer satisfaction.

On the other hand, exploratory factor analysis for these items might also worth a try in hope of finding important psychological construct for further analysis. However, such crazy idea might go far beyond the scope of this homework, so no any related analysis would be conducted in the following study.

Satisfaction vs item scores

```
par(mfrow  = c(3, 5), mar = c(4, 1, 2, 2)+0.1)
for(i in 8:21){
  counts = as.matrix(table(survey1$satisfaction, survey1[,i]), nrow = 2)
  p = counts[2,]/apply(counts, 2, sum)
  se = p*(1-p)/apply(counts, 2, sum)
  plotCI(sort(unique(survey1[,i])), p, ui = p+2*se, li = p-2*se,
           xlab = cols[i], ylab = "prop.of.satisfaction")
}
```

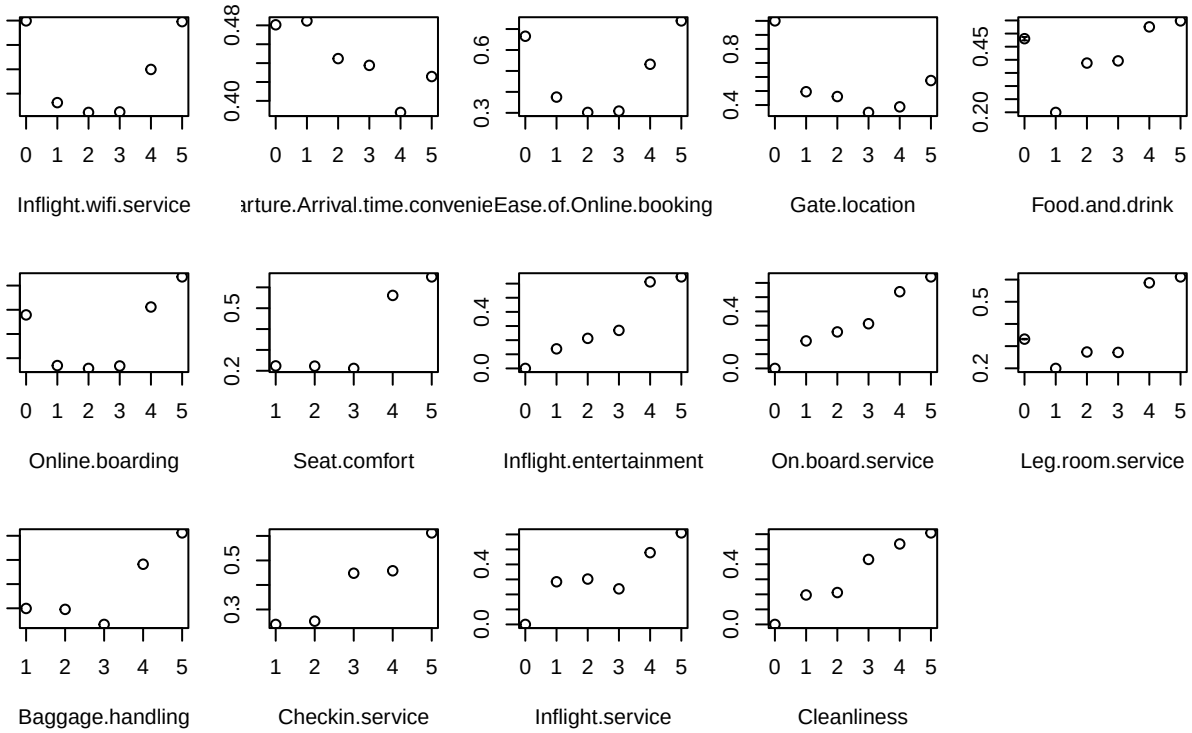

Figure 20: Proportion of satisfaction vs item score

Figure 20 shows the scatter plots, in each of which proportions of satisfaction are plotted against the score for each item. One can see that all of the items seem to have relationship with proportion of satisfaction either linearly or non-linearly.

Satisfaction vs continuous predictors

```
par(mfrow  = c(2, 2), mar = c(3, 4, 4, 2)+0.1)
for(i in c(4, 7, 22, 23)){
  boxplot(survey1[, i]~survey1$satisfaction, xlab = "satisfaction", ylab = cols[i])
}
```

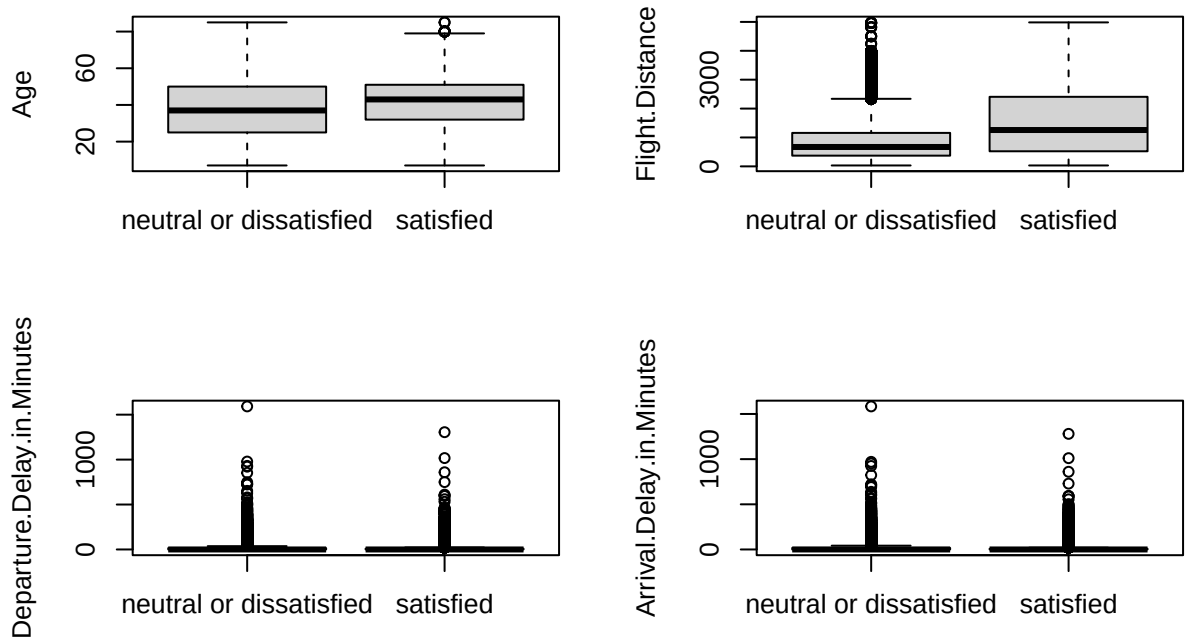

Figure 21: Relation between satisfaction and the continuous variables

Figure 21 shows the relationship between each continuous variable and satisfaction. One can see that customers who were satisfied with the flight did not seem to differ obviously in terms of any of the continuous variable from those who were less satisfied.

Satisfaction vs categorical predictors

```
p.list = list()
j = 1
for(i in c(2, 3, 5, 6)){
  counts = table(survey1[,i], survey1$satisfaction)
  n = apply(counts, 1, sum)
  counts = counts/matrix(rep(apply(counts, 1, sum), 2),
                                ncol = 2)
  counts = melt(counts)
  counts = cbind(counts, sqrt(counts[, 3]*(1-counts[, 3])/rep(n, 2)))
  colnames(counts) = c(cols[i], "satisfaction", "Prop", "se")
  p = ggplot(tail(counts,nrow(counts)/2), aes_string(x=cols[i], y="Prop"))
  p = p + geom_line(size=.6) +
     geom_point(size=.6) +
     geom_errorbar(aes(ymax=Prop+2*se, ymin=Prop-2*se), width=.1)+theme_bw()
  p.list[[j]] = p; j = j + 1
}

ggarrange(p.list[[1]], p.list[[2]], p.list[[3]], p.list[[4]], nrow = 2, ncol = 2)
```

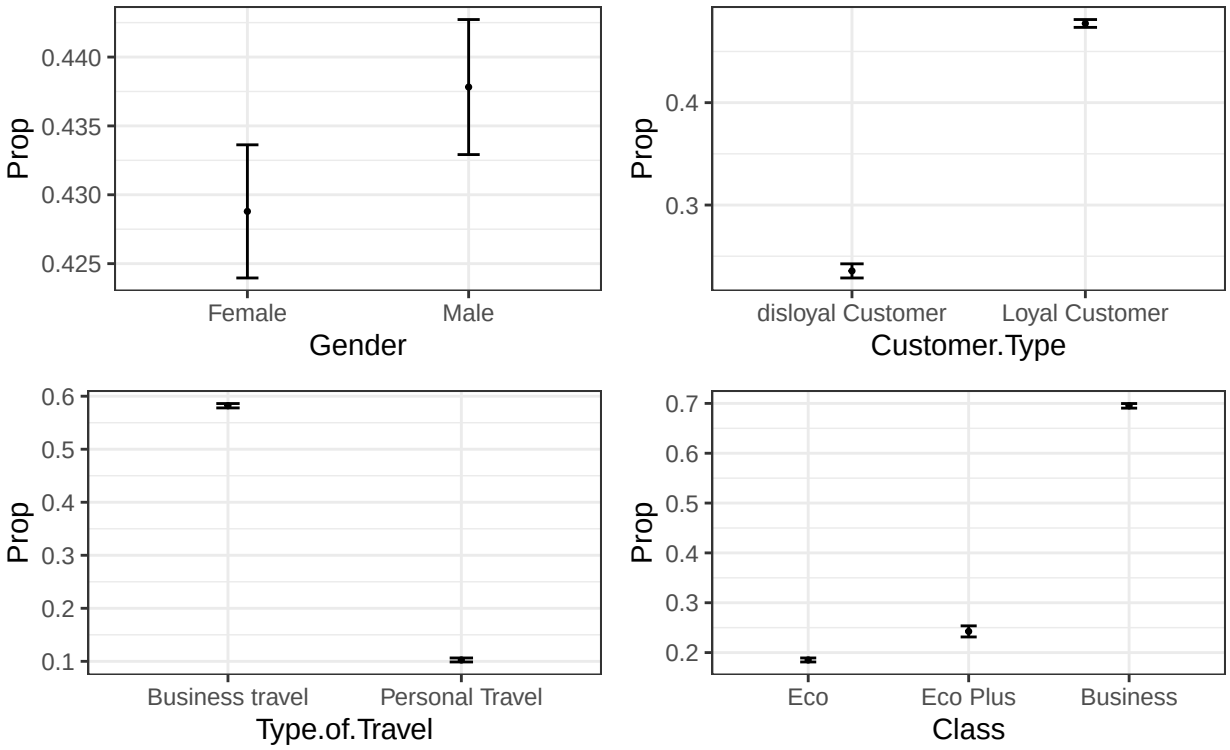

Figure 22: Proportion of satisfaction vs categorical variables

Figure 22 shows the proportion estimate of satisfaction under each category for the categorical variables. One can see that the difference satisfaction proportion seems to be quite significant among the categories for each variable here. The important finding are summarized as follows.

1. Male seemed to be obviously more satisfied with the flight than did Female.
2. Loyal customer seemed to be obviously more satisfied with the flight than did disloyal customer.
3. Customer who took the flight for business travel seemed to be obviously more satisfied than did those who took the flight for personal travel.
4. Customer who bought Business class seemed to be obviously more satisfied with the flight than did those who bought Eco Plus class. Moreover, customer who bought Eco Plus class also seemed to be obviously more satisfied than did those who bought Eco class.

2.3 Find a good model for the classification of customer satisfaction, and summarize the performance.

Since the prediction performance is of the most concern, in the following only random forest (or bagging) and boosting would be considered. For each of the models, 5-fold cross-validation is used to evaluate the test MSE (mean squared error). For more details, cross validation is used to select the tuning parameter (m) for bagging (or random forest), and select the interaction depth (d), shrinkage parameter (λ) and number of trees (B) for boosting. The five folds used for cross-validation is randomly determined as follows.

```
k = 5
set.seed (10)
folds=sample(1:k, nrow(survey1), replace=TRUE)
folds.n = table(folds)
```

According to the preliminary findings in EDA, the 14 item scores, gender, customer type, type of travel and class are considered as the important predictors. Hence, they are used in the following modeling procedure.

a. Random forest

Throughout the cross-validation procedure, the tuning parameter m is searched among 2 to 7.

```
params = 2:7
if("rfcv.error.RData" %in% list.files()){
  source("rfcv.error.RData")
}else{
  cv.errors=matrix(NA, k, length(params))
  for(i in 1:length(params)){
    param = params[i]
    for(j in 1:k){
        fit = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                           +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                           +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                           +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                           +Customer.Type+Age+Type.of.Travel, data = survey1, subset = folds != j,
                           split = "deviance", mtry = param)
        pred = predict(fit, survey1[folds == j,])
        confuse = table(survey1[folds == j,]$satisfaction, pred)
        acc = sum(diag(confuse))/sum(confuse)
        cv.errors[j, i] = acc
    }
  }
  dump("cv.errors", "rfcv.error.RData")
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(survey1)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "m", ylab = "test Accuracy")
opt.m = params[which.max(cv)]
abline(v = params[which.max(cv)], col = "red")
legend("topleft", c("test Accuracy", "CI"), lty = 1:2)
```

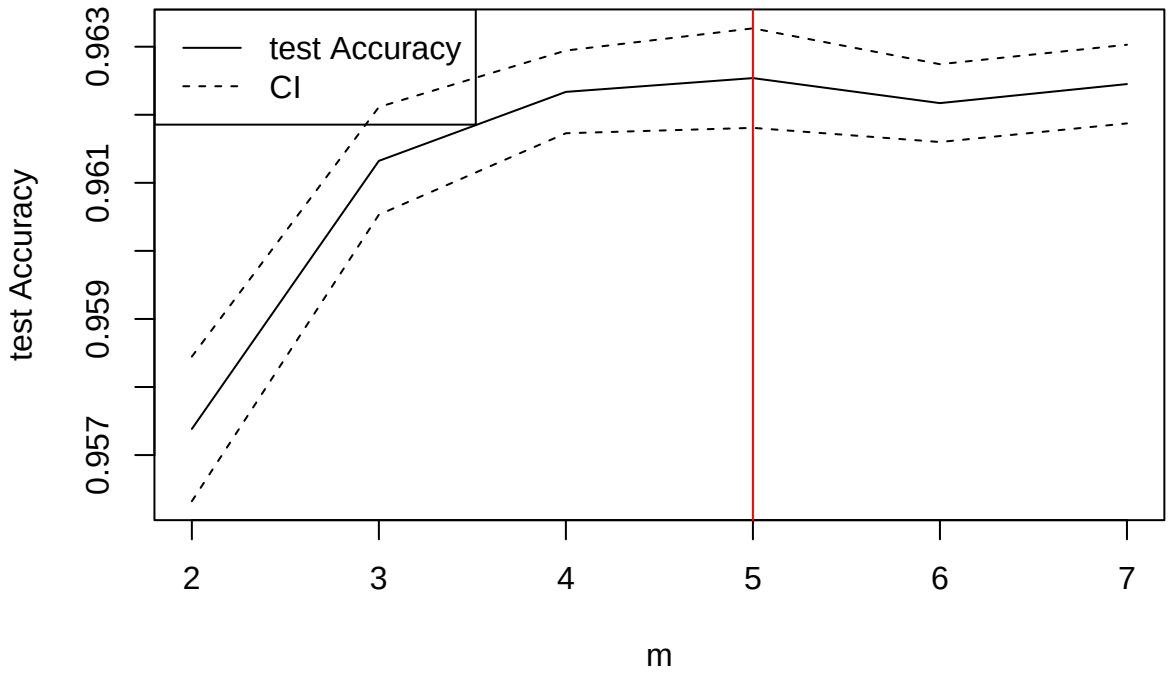

Figure 23: Cross validation for the depth and shrinkage parameter

Figure 23 suggests that it might be better to set m at 5 for the highest accuracy estimate. This result also implies that bagging, which use m equal to the number of all predictors, might not be a good choice. We thus fit again the random forest under m = 5 using the whole training data.

```
if("rf2.RData" %in% list.files()){
  source("rf2.RData")
}else{
  rf2 = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                           +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                           +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                           +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                           +Customer.Type+Age+Type.of.Travel, data = survey1,
                           split = "deviance", mtry = opt.m)
  dump("rf2", "rf2.RData")
}
plot(rf2, main = "")
```

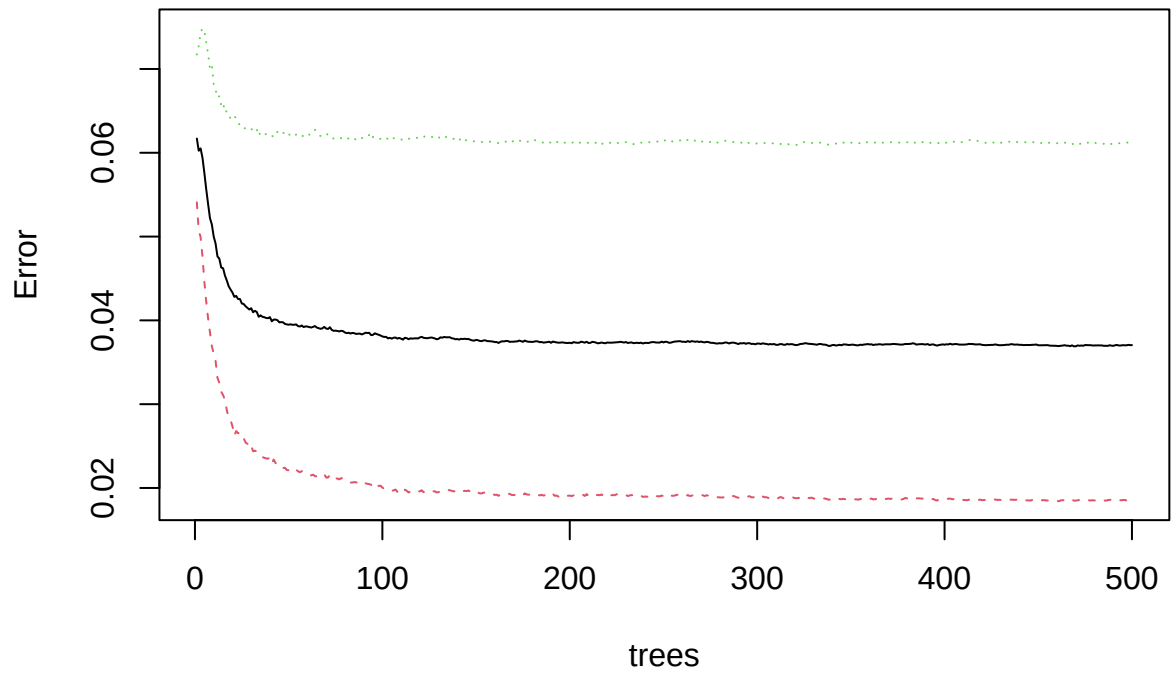

Figure 24: Cross validation for the number of trees

Figure 24 shows that 400 trees seems to be good enough. Hence, we refit the random forest under ntree = 400.

```
final.rf2 = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                         +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                         +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                         +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                         +Customer.Type+Age+Type.of.Travel, data = survey1,
                         split = "deviance", mtry = opt.m, n.tree = 400)
# importance(rf)
varImpPlot(final.rf2, main = "")
```

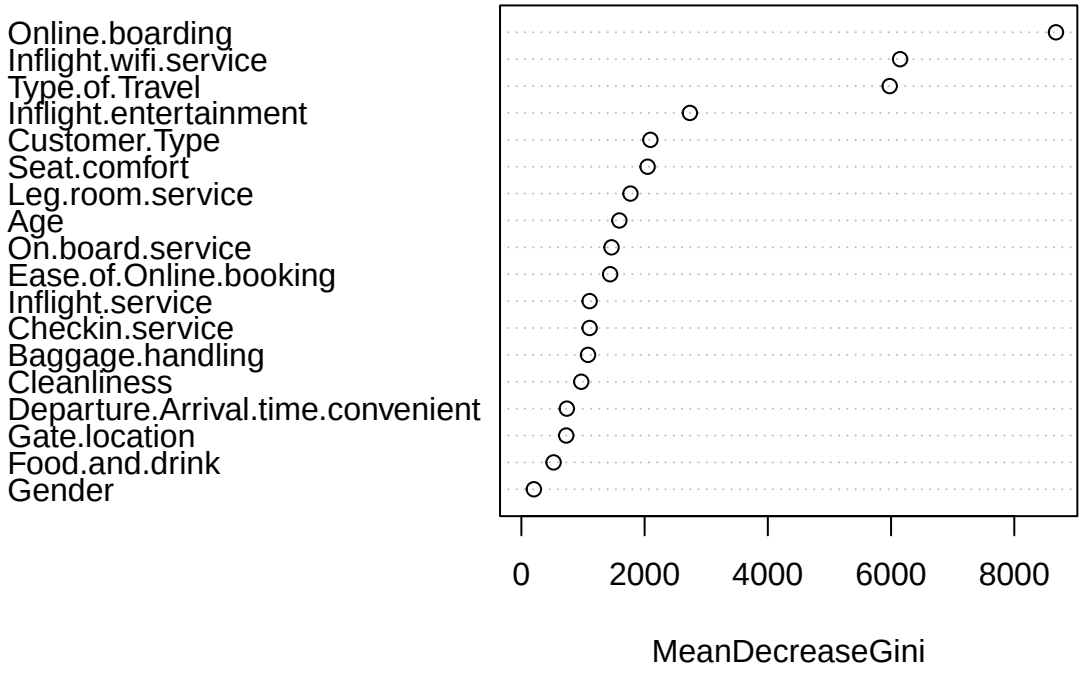

Figure 25: Important plot for predictors

Identify important predictors

Figure 25 shows that Online.boarding, Inflight.wifi.service and Type.of.Travel might be the most important predictor as they seem to reduce the most Gini index on average.

b. Boosting

Throughout the cross-validation procedure, the interaction depth (d) is searched among 5 to 8, while the shrinkage parameter (λ) is searched among 0.005 to 0.05 in interval of 0.005. After d and λ are decided, the number of tree (B) would be tuned afterwards by conditioning on the optimal d and λ.

```
response = ifelse(survey1$satisfaction == "neutral or dissatisfied", 0, 1)
survey1$response = response
btcv = function(ntree){
    cv.errors = array(NA, dim = c(k, length(params1), length(params2)))
    for(i in 1:length(params1)){
      param1 = params1[i]
      for(l in 1:length(params2)){
        print(c(i, l))
        param2 = params2[l]
         for(j in 1:k){
          fit = gbm(response~Inflight.wifi.service+Departure.Arrival.time.convenient
                          +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                          +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                          +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
```

```
                          +Customer.Type+Age+Type.of.Travel, data = survey1[folds != j,],
                           distribution  = "bernoulli", n.trees = ntree,
                           interaction.depth=param1, shrinkage = param2, verbose = FALSE)
          pred = predict(fit, survey1[folds == j,], type = "response")
          pred = ifelse(pred > 0.5, 1, 0)
          confuse = table(survey1[folds == j,]$response, pred)
          acc = sum(diag(confuse))/sum(confuse)
          cv.errors[j, i, l] = acc
        }
      }
    }
    return(cv.errors)
}
params1 = 5:8 # d
params2 = seq(0.005, 0.05, 0.005) # lambda
if("bscv.error2.RData" %in% list.files()){
  source("bscv.error2.RData")
}else{
  cv.errors = btcv(800)
  dump("cv.errors", "bscv.error2.RData")
}
cv = apply(cv.errors*array(matrix(rep(folds.n, length(params1)), nrow = k),
                           dim = c(k, length(params1), length(params2))),
                           c(2, 3), sum)/nrow(survey1)
rownames(cv) = params1; colnames(cv) = params2
pheatmap(cv, cluster_rows = FALSE, cluster_cols = FALSE, main = "test Accuracy")
```

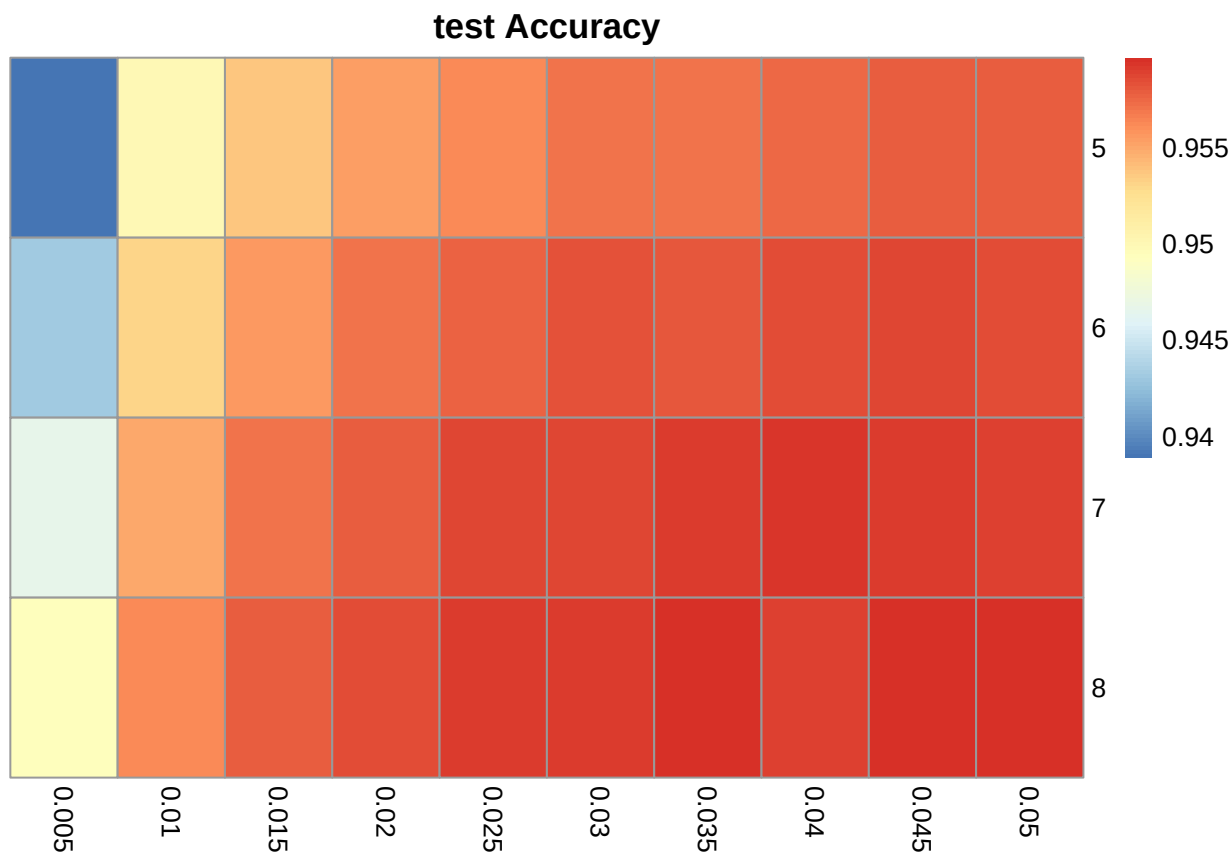

Figure 26: Cross validation for the depth and shrinkage parameter

```
opt.params = c(params1[ifelse(which.max(cv) %% 4 == 0, 4, which.max(cv) %% 4)],
            params2[ifelse(which.max(cv) %% 4 == 0, which.max(cv) %/% 4,
                           which.max(cv) %/% 4 + 1)])
```

Figure 26 displays the test MSE estimates obtained under each setting (pair) of the parameters d and λ. Each row of the matrix shown refers to one distinct value of d, and each of the column refers to one distinct value of λ. It turns out that d = 8 and $\lambda = 0.05$ might provide the highest test accuracy. (One might argue that the optima found is merely a local one since it is a boundary point. However, according to the increasing trend displayed in Figure 26, it is reasonable to believe that the optima should already be very closed to the global one.)

```
params = seq(250, 5000, 250)
if("ntree.cv2.RData" %in% list.files()){
  source("ntree.cv2.RData")
}else{
  cv.errors=matrix(NA, k, length(params))
  for(i in 1:length(params)){
    print(params[i])
    for(j in 1:k){
          fit = gbm(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                          +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                          +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                          +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
```

```
                          +Customer.Type+Age+Type.of.Travel, data = survey1[folds != j,],
                           distribution= "deviance", n.trees = ntree,
                           interaction.depth=param1, shrinkage = param2, verbose = FALSE)
          pred = predict(fit, survey1[folds == j,])
          confuse = table(survey1[folds == j,]$satisfaction, pred)
          acc = sum(diag(confuse))/sum(confuse)
          cv.errors[j, i] = acc
        }
  }
  dump("cv.errors", "ntree.cv2.RData")
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(bike)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "trees", ylab = "test MSE")
opt.ntree = params[which.max(cv)]
abline(v = opt.ntree, col = "red")
legend("topleft", c("test Accuracy", "CI"), lty = 1:2)
```

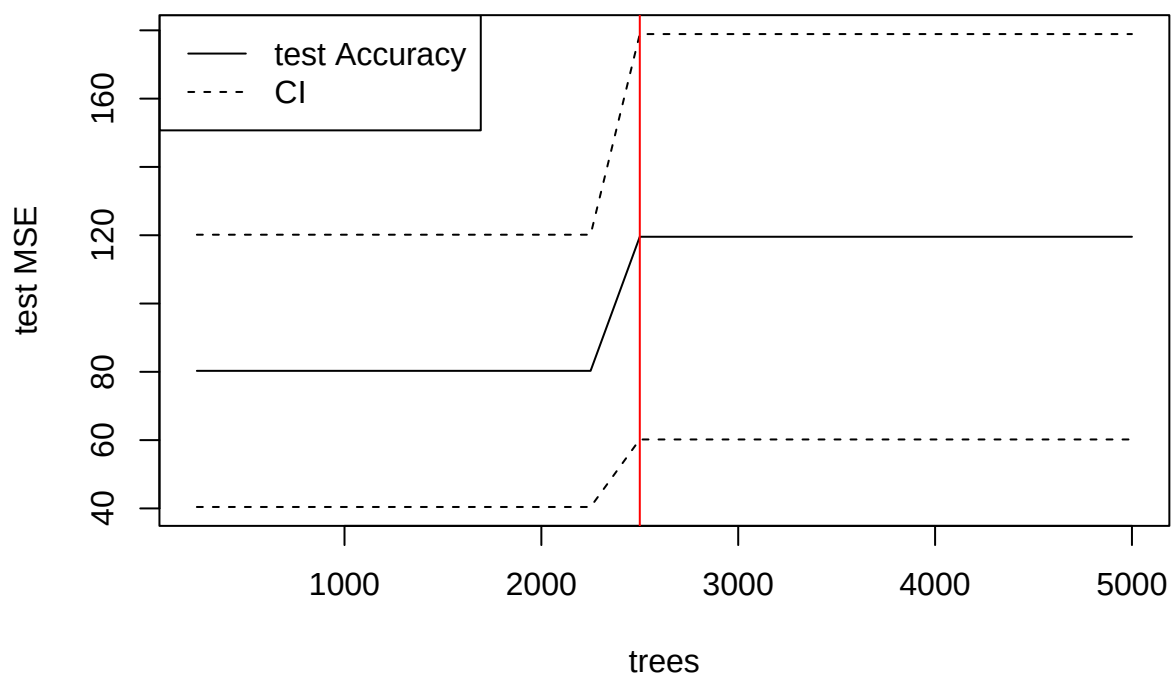

Figure 27: Cross validation for the number of trees

Figure 27 suggests that it might be better to set B at 2500 for the highest test accuracy. Finally, we fit again the boosting model by such optimal setting (including d, λ and B) using the whole training data.

```
boost2 = gbm(response~Inflight.wifi.service+Departure.Arrival.time.convenient
                          +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                          +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                          +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                          +Customer.Type+Age+Type.of.Travel,
                          data = survey1, distribution = "bernoulli",n.trees = opt.ntree,
                          interaction.depth = opt.params[1], shrinkage = opt.params[2])
imp = summary(boost2, plotit = FALSE)
imp = imp[order(imp$rel.inf),]
barplot(imp$rel.inf, horiz = TRUE, names.arg = imp$var, las = 2,
        col = cbp[6], cex.names = 0.7, xlab = "relative influence")
```

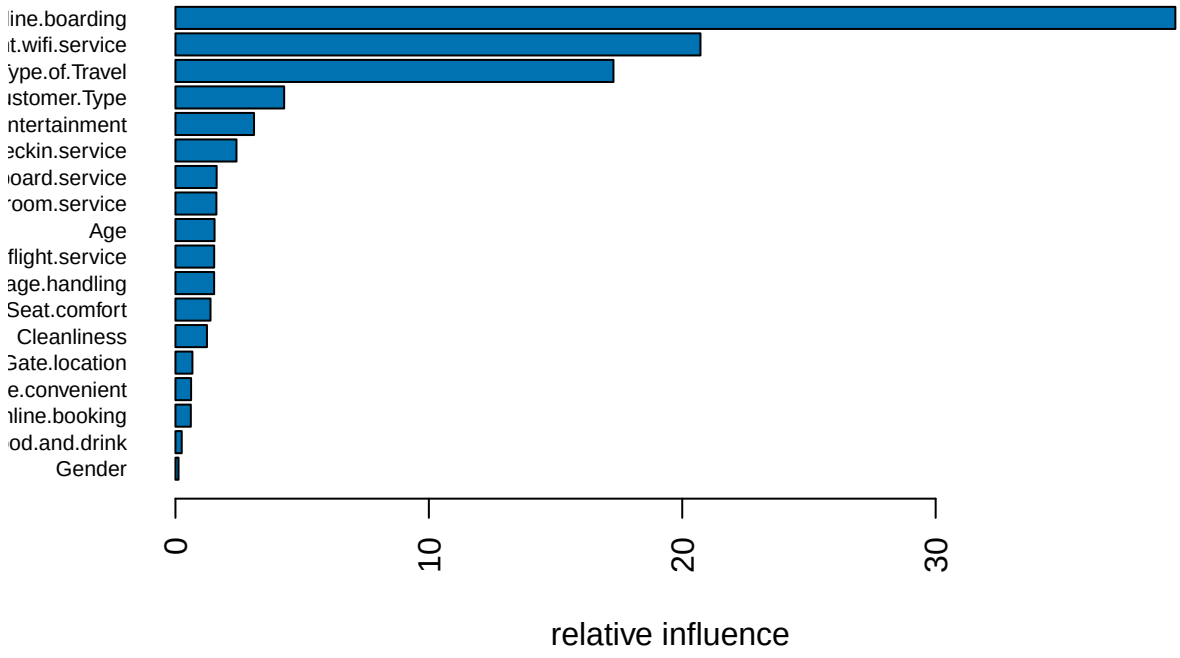

Figure 28: Important plot for predictors

Identify important predictors

Figure 28 shows the importance of predictors used in the boosting. From top to down, one might conclude that Online.boarding, Inflight.wifi.service and Type.of.Travel seem to be relatively more important predictors than the others.

2.4 Evaluate the prediction performance via testing error on testing set

```r
test.summary = matrix(NA, nrow = 1, ncol = 2)
colnames(test.summary) = c("bagging", "boosting")
rownames(test.summary) = c("Accuracy")
models = list(final.rf2, boost2)
# par(mfrow = c(2, 3), mar = c(5, 4.5, 4, 2) + 0.1)
for(i in 1:2){
  fit = models[[i]]
  if(i == 1){
    pred = predict(fit, survey1.test)
    confuse = table(survey1.test$satisfaction, pred)
    acc = sum(diag(confuse))/sum(confuse)
  }else if(i == 2){
    pred = predict(fit, survey1[folds == j,], type = "response")
    pred = ifelse(pred > 0.5, 1, 0)
    confuse = table(survey1[folds == j,]$response, pred)
    acc = sum(diag(confuse))/sum(confuse)
  }
  test.summary[,i] = acc
}
```

```r
kable(as.data.frame(test.summary), booktabs = TRUE, linesep = "",
      align = "c", caption = "Summary of testing error for each model") %>%
  kable_styling(latex_options = c("hold_position"))
```

Table 12: Summary of testing error for each model

	bagging	boosting
Accuracy	0.9602297	0.9696052

Table 12 shows the test accuracy of each model, suggesting that boosting might provide the higher accuracy. However, their difference does not seem obvious in this case.

2.5 Identify the important aspects of the flight experience related to customer satisfaction (or dis-satisfaction). Based on your findings, give some specific suggestions (provided with the data evidence) to the airline company for achieving a higher customer satisfaction.

Online.boarding, Inflight.wifi.service and Type.of.Travel are regarded as the most important predictors both in the random forest and the boosting. Such result is quite consistent with the findings in EDA.

More important details are summarized as follows. Note that the first and the second one is regarding to flight experience, and the last one is regarding to type of travel.

1. There seems to be two kinds of customers. One kind of them consists of customers who give positive remark regarding to the online boarding. Such kind of customers have high satisfaction toward the flight. The other kind consists of customers who answer 0 to the question regarding to online boarding. This kind of people also have high satisfaction toward the flight but without clear reason since we are not quite sure about the meaning of 0.

2. There also seems to be two kinds of customers. One kind of them consists of customers who give positive remark regarding to the inflight wifi service. Such kind of customers have high satisfaction toward the flight. The other kind consists of customers who answer 0 to the question regarding to inflight wifi service. This kind of people also have high satisfaction toward the flight but without clear reason since we are not quite sure about the meaning of 0.

3. It seems that Business traveler tends to be more satisfied with the flight than personal traveler does since higher proportion of the customers who took business travel responded "satisfied" than did those who took personal travel.

In conclusion, we suggest the airline company to invest more on the online boarding system and inflight wifi service. Several possible measures are listed as follows.

1. Enhance the accessibility of related system such as delivering the mobile phone app version of the online boarding system.

2. Improve the user interface of the related system so that it may become more user-friendly to the elderly.

3. Conduct interview study with personal travelers who may not be so satisfied with the flight for more valuable suggestion.

2.6 Add some noise factors (both continuous variable and categorical variable) in your data set (call these added variables z1-z?), and check the ranking of variable importance of these z-variables relative to the original meaningful input variables. Make your comments regarding this experiment.

z1 and z2 are generated independently of the data, where z1 is generated according to normal distribution with mean 500 and standard deviation 400 and z2 is a binary variable generated according to bernoulli distribution with success probability 0.3. The same procedure including cross validation and the final model fitting are conducted in the same manner as before for random forest and boosting.

```
z1 = rnorm(nrow(survey1), mean = 500, sd = 400)
z2 = rbinom(nrow(survey1), size = 1, p = 0.3)
survey1$z1 = z1; survey1$z2 = factor(z2, levels = c("0", "1"))
```

a. Random forest

Throughout the cross-validation procedure, the tuning parameter m is searched among 2 to 15.

```
params = 2:15
if("rfcv.error2.RData" %in% list.files()){
  source("rfcv.error2.RData")
}else{
  cv.errors=matrix(NA, k, length(params))
  for(i in 1:length(params)){
    print(i)
    param = params[i]
    for(j in 1:k){
        fit = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                           +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                           +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                           +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                           +Customer.Type+Age+Type.of.Travel+z1+z2, data = survey1, subset = folds != j
```

```
                              split = "deviance", mtry = param)
        pred = predict(fit, survey1[folds == j,])
        confuse = table(survey1[folds == j,]$satisfaction, pred)
        acc = sum(diag(confuse))/sum(confuse)
        cv.errors[j, i] = acc
    }
  }
  dump("cv.errors", "rfcv.error2.RData")
}
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(survey1)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "m", ylab = "test Accuracy")
opt.m = params[which.max(cv)]
abline(v = params[which.max(cv)], col = "red")
legend("topleft", c("test Accuracy", "CI"), lty = 1:2)
```

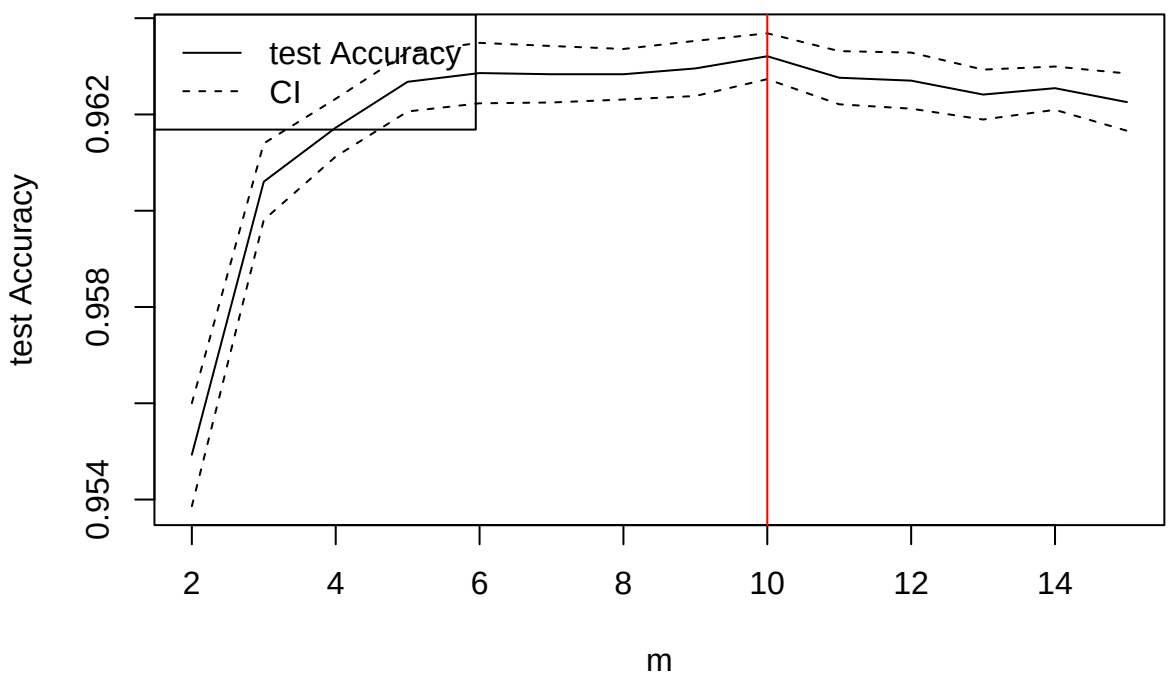

Figure 29: Cross validation for the tuning parameter (m)

Figure 29 suggests that it might be better to set m at 10 for the highest accuracy estimate. This result also implies that bagging, which use m equal to the number of all predictors, might not be a good choice. We thus fit again the random forest under m = 10 using the whole training data.

```
if("rf3.RData" %in% list.files()){
  source("rf3.RData")
}else{
  rf3 = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                           +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                           +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                           +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                           +Customer.Type+Age+Type.of.Travel+z1+z2, data = survey1,
                           split = "deviance", mtry = opt.m)
  dump("rf3", "rf3.RData")
}
plot(rf3, main = "")
```

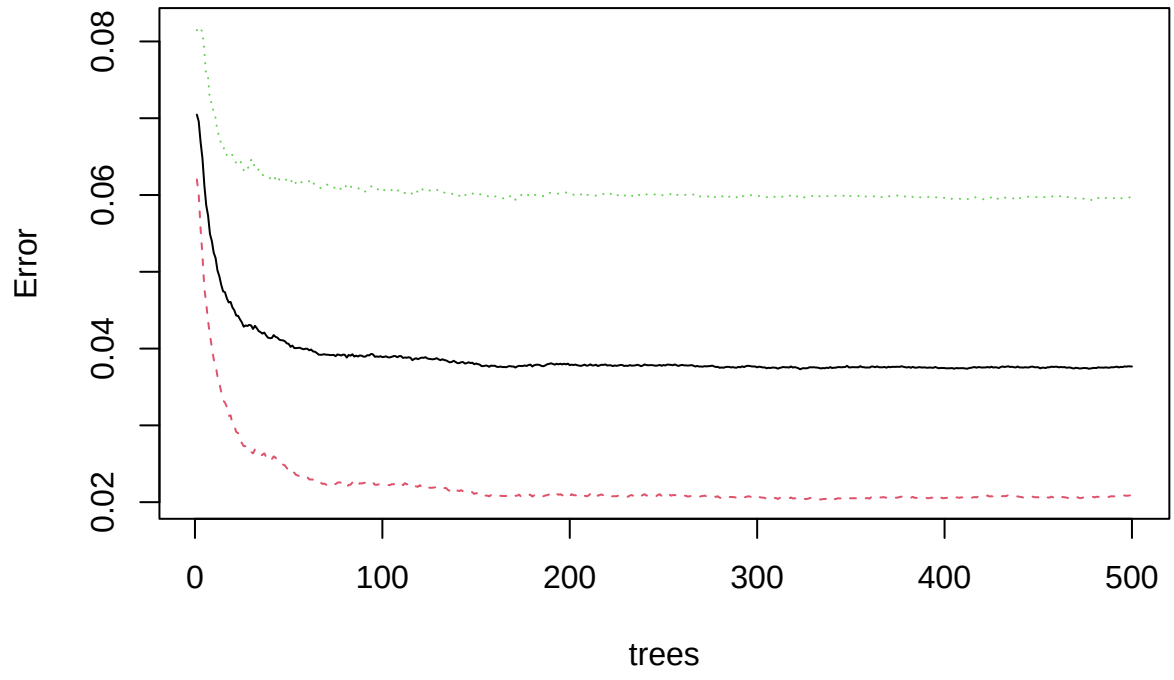

Figure 30: Cross validation for the number of trees

Figure 30 suggests that 200 trees might be good enough. Hence, we refit the random forest under ntree = 200.

```
final.rf3 = randomForest(satisfaction~Inflight.wifi.service+Departure.Arrival.time.convenient
                           +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                           +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                           +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                           +Customer.Type+Age+Type.of.Travel+z1+z2, data = survey1,
                           split = "deviance", mtry = opt.m, n.tree = 200)
```

```
varImpPlot(final.rf3)
```

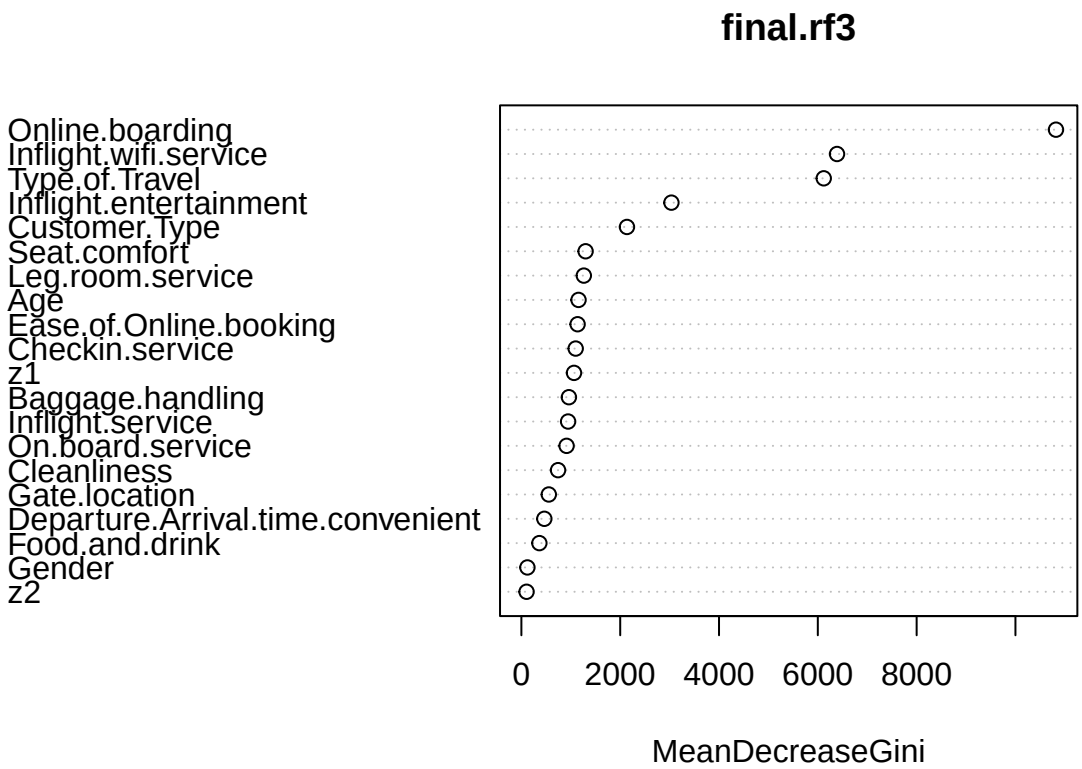

Figure 31: Cross validation for the tuning parameter (m)

Figure 31 shows that z1 which is arbitrarily added into the model are mis-ranked in the very front place (rank 11). such result is not reasonable as we have seen that there should be more than 10 predictors that are more important than z1 according to the findings in EDA. The reason why such result may happen might be due to the random disturbances carried by z1 and the lack of validity of the importance measure used here. As for z2, it is ranked as the last important one, which is in consistent with the fact that it is an arbitrary and meaningless variable.

b. Boosting

Here we fit the boosting with setting of tuning parameters that have been used in the last section, that is, d = 8, $\lambda = 0.05$ and ntree = 2500.

```
boost3 = gbm(response~Inflight.wifi.service+Departure.Arrival.time.convenient
                          +Ease.of.Online.booking+Gate.location+Food.and.drink+Online.boarding
                          +Seat.comfort+Inflight.entertainment+On.board.service+Leg.room.service
                          +Baggage.handling+Checkin.service+Inflight.service+Cleanliness+Gender
                          +Customer.Type+Age+Type.of.Travel+z1+z2,
                          data = survey1, distribution = "bernoulli",n.trees = opt.ntree,
                          interaction.depth = opt.params[1], shrinkage = opt.params[2])
imp = summary(boost3, plotit = FALSE)
imp = imp[order(imp$rel.inf), ]
```

```
barplot(imp$rel.inf, horiz = TRUE, names.arg = imp$var, las = 2,
        col = cbp[6], cex.names = 0.4, xlab = "relative influence")
```

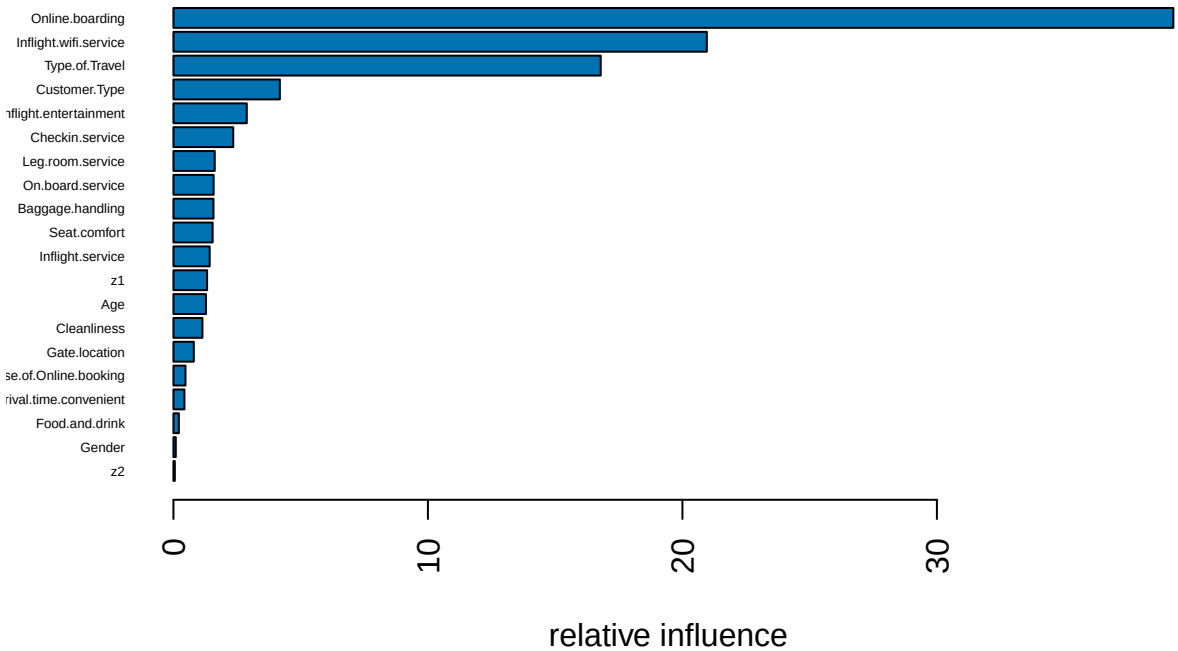

Figure **??** shows that z1 and z2 are ranked very behind but still they are not the least important. It is still not a good news since according to the results of EDA which suggest some importance of those candidate predictors, z1 and z2 should be ranked as the last two.

3 Appendix

3.1 A. Summary for the variables in Problem 2

```
count(survey1$Gender, survey1$Customer.Type, cols[2], cols[3])
```

Table 13: Summary for Gender and Customer.Type

	Gender		Customer.Type	
	Female	Male	disloyal Customer	Loyal Customer
counts	41920	40955	15120	67755

```
count(survey1$Type.of.Travel, survey1$Class, cols[5], cols[6])
```

Table 14: Summary for Type.of.Travel and Class

	Type.of.Travel		Class		
	Business travel	Personal Travel	Eco	Eco Plus	Business
counts	57157	25718	37278	5947	39650

```
count(survey1$Inflight.wifi.service, survey1$Departure.Arrival.time.convenient, cols[8], cols[9])
```

Table 15: Summary for Inflight.wifi.service and Departure.Arrival.time.convenient

	Inflight.wifi.service						Departure.Arrival.time.convenient					
	0	1	2	3	4	5	0	1	2	3	4	5
counts	2494	14166	20677	20597	15709	9232	4251	12379	13670	14328	20310	17937

```
count(survey1$Ease.of.Online.booking, survey1$Gate.location, cols[10], cols[11])
```

Table 16: Summary for Ease.of.Online.booking and Gate.location

	Ease.of.Online.booking						Gate.location					
	0	1	2	3	4	5	0	1	2	3	4	5
counts	3589	13935	19227	19461	15569	11094	1	13898	15500	22769	19583	11124

```
count(survey1$Food.and.drink, survey1$Online.boarding, cols[12], cols[13])
```

Table 17: Summary for Food.and.drink and Online.boarding

	Food.and.drink						Online.boarding					
	0	1	2	3	4	5	0	1	2	3	4	5
counts	77	10198	17553	17784	19415	17848	1933	8486	14051	17387	24478	16540

```
count(survey1$Seat.comfort, survey1$Inflight.entertainment, cols[14], cols[15])
```

```
count(survey1$On.board.service, survey1$Leg.room.service, cols[16], cols[17])
```

```
count(survey1$Baggage.handling, survey1$Checkin.service, cols[18], cols[19])
```

Table 18: Summary for Seat.comfort and Inflight.entertainment

	Seat.comfort					Inflight.entertainment					
	1	2	3	4	5	0	1	2	3	4	5
counts	9598	11909	14902	25289	21177	12	9860	14095	15274	23421	20213

Table 19: Summary for On.board.service and Leg.room.service

	On.board.service						Leg.room.service					
	0	1	2	3	4	5	0	1	2	3	4	5
counts	3	9426	11716	18243	24565	18922	374	8150	15652	15993	22988	19718

Table 20: Summary for Baggage.handling and Checkin.service

	Baggage.handling					Checkin.service				
	1	2	3	4	5	1	2	3	4	5
counts	5734	9158	16511	29798	21674	10279	10226	22642	23303	16425

```
count(survey1$Inflight.service, survey1$Cleanliness, cols[20], cols[21])
```

Table 21: Summary for Inflight.service and Cleanliness

	Inflight.service						Cleanliness					
	0	1	2	3	4	5	0	1	2	3	4	5
counts	3	5562	9135	16229	30236	21710	9	10556	12905	19591	21691	18123

```
counts = t(data.frame(c(table(survey1$satisfaction))))
rownames(counts) = "counts"
header = c(1,length(names(table(survey1$satisfaction))))
header = header[header != 0]
names(header) = c(" ", cols[24])
kable(counts, format = "latex", booktabs = TRUE,
  col.names = c(names(table(survey1$satisfaction))),
  align = "c", caption = paste("Summary for", cols[24]), escape = FALSE
  ) %>%
  kable_styling(latex_options = c("hold_position")) %>%
  add_header_above(header)
```

```
stat = round(sapply(survey1[c(4, 7, 22, 23)], summary),2)
kable(as.data.frame(stat), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE, linesep = "", caption =  "Summary for the continuous variables") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

Table 22: Summary for satisfaction

	satisfaction	
	neutral or dissatisfied	satisfied
counts	46969	35906

Table 23: Summary for the continuous variables

	Age	Flight.Distance	Departure.Delay.in.Minutes	Arrival.Delay.in.Minutes
Min.	7.00	31.0	0.00	0.00
1st Qu.	27.00	414.0	0.00	0.00
Median	40.00	843.0	0.00	0.00
Mean	39.41	1189.7	14.73	15.18
3rd Qu.	51.00	1744.0	12.00	13.00
Max.	85.00	4983.0	1592.00	1584.00